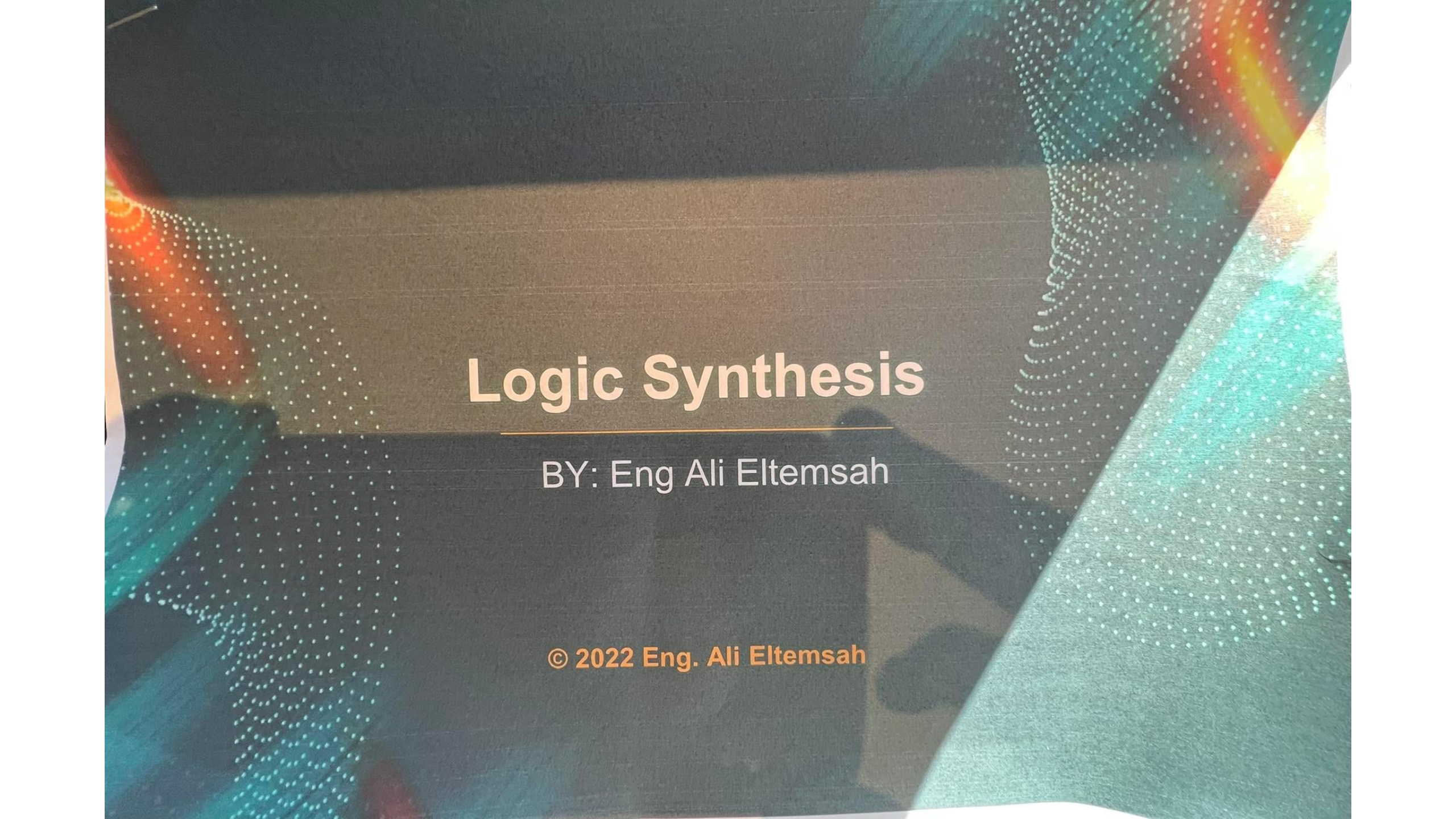




Logic Synthesis

BY: Eng Ali Eltemsah

© 2022 Eng. Ali Eltemsah

- Introduction to Synthesis
- Libraries Definition
 - .LIB
 - .LEF
 - .GDS
- Operating Condition
 - Worst-Case Library
 - Best-Case Library
 - Typical Library
- Reading Design Files
- Linking
- Compile
- Generate Gate Level Netlist
- Timing Analysis
 - Dynamic Timing Analysis
 - Static Timing Analysis
- Design Environment Constraints
 - Set_load
 - Set_driving_cell
 - Set_operating_condition
 - Set_wireload_model
- Design Optimization Constraints
 - Create_clock
 - Set_clock_latency
 - Set_clock_uncertainty
 - Set_clock_transition
 - Set_input_delay
 - Set_output_delay
 - Set_max_area
 - Set_max_dynamic_power
 - Set_dont_touch_network
- Timing Exceptions
 - False Paths
 - Multicycle Paths
- Clock Grouping
- Reporting
 - Area
 - Power
 - Timing

Introduction

• What is Logic Synthesis?

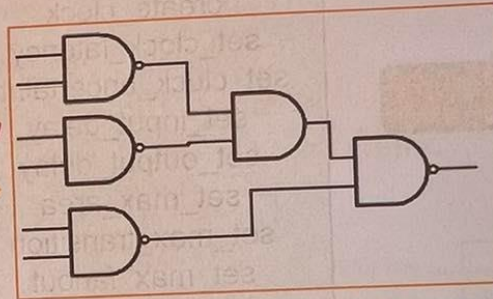
- Synthesis is the process that translate and map RTL code written in HDL into a technology dependent gate-level netlist, optimized for a set of pre-defined constraints.

- Synthesis = Translation + **Mapping** + Optimization

```
assign CarryOut = tmp[8]; // Carryout flag
always @(*)
begin
  case(ALU_Sel)
    4'b0000: // Addition
      ALU_Result = A + B ;
```

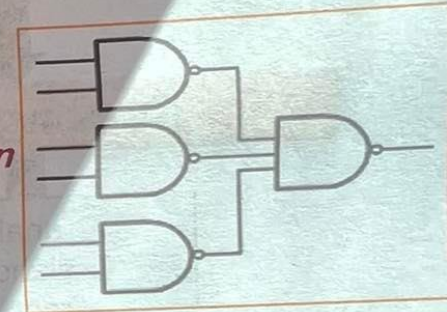
Hardware Description Language

Translation



GTECH (Generic TECH)
No Timing Info

Mapping
&
Optimization



Target Technology
Timing Info Exists

• You start with:

- A behavioral RTL design
- A standard cell library
- A set of design constraints

synthesis tool
inputs

• You finish with:

- Tech mapped gate-level netlist
- Timing, Area and Power reports

synthesis tool
outputs

```
module counter(
  input clk, rstn, load,
  input [1:0] in,
  output reg [1:0] out);
always @(posedge clk)
  if (!rstn) out <= 2'b0;
  else if (load) out <= in;
  else out <= out + 1;
endmodule
```

Synthesis

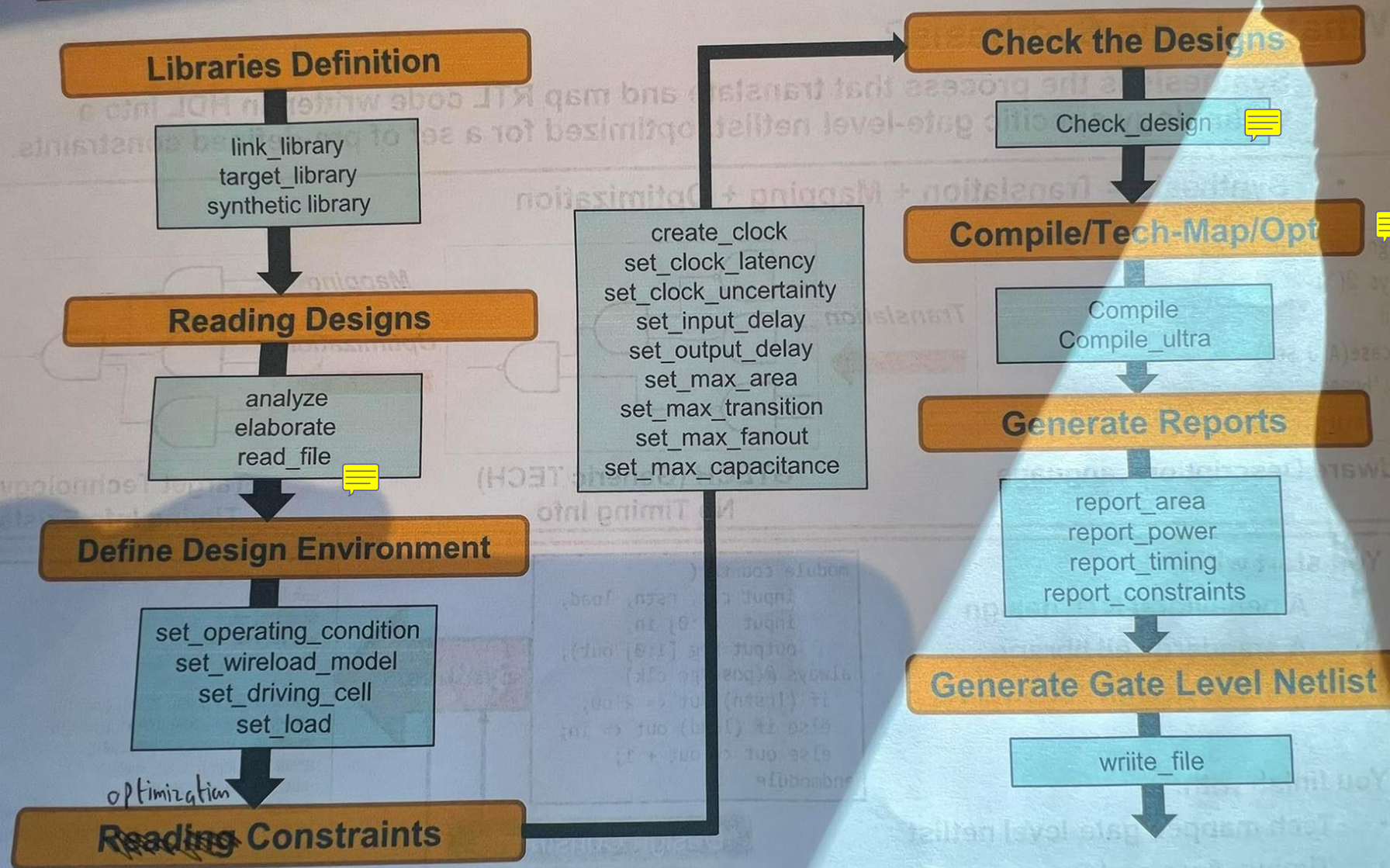
Design Constraints

Standard Cell Library

```
module counter ( clk, rstn, load, in, out );
  input [1:0] in;
  output [1:0] out;
  input clk, rstn, load;
  wire N6, N7, n5, n6, n7, n8;

  HD0FFPQ1 out_reg_1 (.D(N7),.CK(clk),.Q(out[1]));
  HD0FFPQ1 out_reg_0 (.D(N6),.CK(clk),.Q(out[0]));
  HDNANZDL U8 (.A1(out[0]),.A2(n5),.Z(n8));
  HDNANZDL U9 (.A1(n5),.A2(n7),.Z(n6));
  HDINVOL U10 (.A(load),.Z(n5));
  HD0A211DL U11 (.A1(in[0]),.A2(n5),.B(rstn),.C(n8),.Z(N6));
  HD0A211DL U12 (.A1(in[1]),.A2(n5),.B(rstn),.C(n6),.Z(N7));
  HD0KNOR2DL U13 (.A1(out[1]),.A2(out[0]),.Z(n7));
endmodule
```


Synthesis Flow

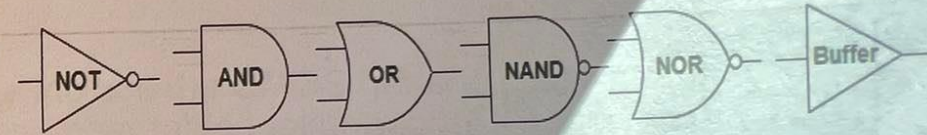


Libraries Definition

What is Technology (Standard Cell) Library?

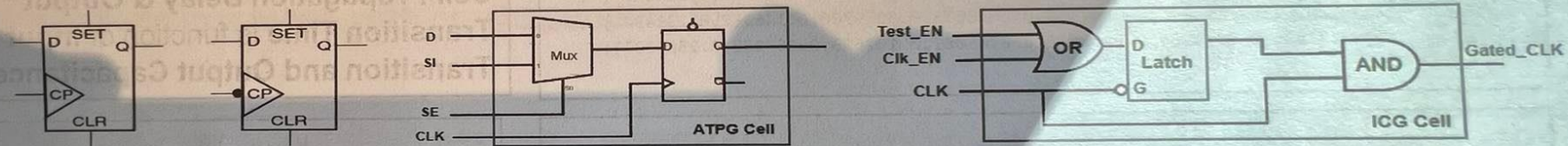
- It is a collection of digital logic cells:-

1. Combinational logic gates such as AND, NAND, OR, NOR, Inverters and Buffers with different sizes to provide different driving strengths.



2. Sequential cells as flip-flops and latches:-

- Positive/Negative Edge Triggered Flip Flops.
- Synchronous/Asynchronous Reset/Set Flip Flop
- Clock Gating Cells : ICG (Integrated Clock Gating) Cells.
- ATPG Cells: Scan-enabled Flip Flops



- Standard cell library is delivered by the fab with a collection of files that provide all the information needed by the various EDA tools, so it is provided in 3 different views:-

- Behavioral view

1. Vital (Verilog/VHDL) Library (std_cell.v/std_cell.vhd) used in simulation (GLS)

- Liberty Timing view

1. Liberty Timing file (std_cell.lib) which characterized with cells area, power as well as timing info used for Static Timing Analysis, so mainly it provide the following info:-

- Cell Name
- Cell Function
- Cell Internal/leakage Power
- Cell Area
- Cell outputs rise/fall transition
- rise/fall propagation delays

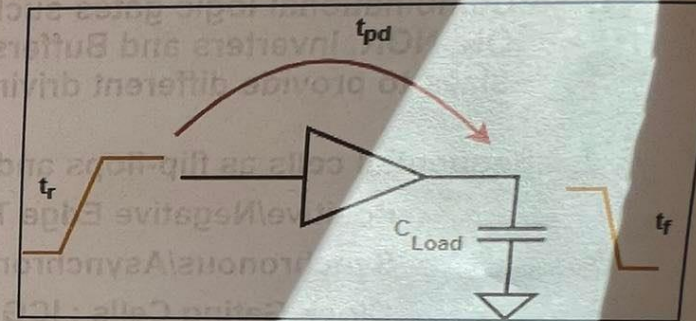
Libraries Definition

.LIB File

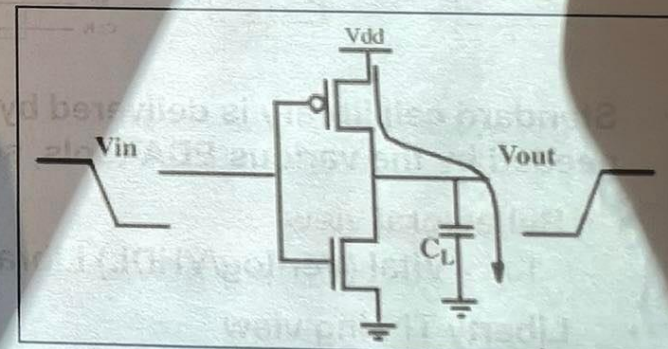


```
timing () {  
    related_pin      : "A1";  
    timing_sense     : negative_unate;  
  
    cell_fall(Timing_7_7) {  
        index_1 ("0.00117378,0.00472397,0.0171859,0.0409838,0.0780596,0.130081,0.198535");  
        index_2 ("0.365616,1.823900,3.647810,7.295610,14.591200,29.182500,58.364900");  
        values ("0.0106270,0.0150189,0.0204521,0.0312612,0.0528211,0.0959019,0.182032", \  
                "0.0116171,0.0160692,0.0215549,0.0324213,0.0540285,0.0971429,0.183289", \  
                "0.0157475,0.0207077,0.0261030,0.0369216,0.0585239,0.101654,0.187820", \  
                "0.0193780,0.0263217,0.0337702,0.0462819,0.0677259,0.110616,0.196655", \  
                "0.0218025,0.0305247,0.0399593,0.0560603,0.0822203,0.125293,0.210827", \  
                "0.0229784,0.0334449,0.0447189,0.0640615,0.0959700,0.146382,0.231434", \  
                "0.0227986,0.0349768,0.0480836,0.0705081,0.107693,0.167283,0.259623");  
    }  
}
```

```
internal_power () {  
    related_pin      : "A1";  
    fall_power(Power_7_7) {  
        index_1 ("0.00117378,0.00472397,0.0171859,0.0409838,0.0780596,0.130081,0.198535");  
        index_2 ("0.365616,1.823900,3.647810,7.295610,14.591200,29.182500,58.364900");  
        values ("0.523620,0.538965,0.551079,0.556548,0.561151,0.564018,0.564418", \  
                "0.459570,0.484698,0.509668,0.529672,0.543887,0.554682,0.559331", \  
                "0.434385,0.457202,0.470452,0.498312,0.517651,0.538469,0.550091", \  
                "0.728991,0.630651,0.581024,0.559124,0.551408,0.553714,0.557387", \  
                "1.306597,1.153240,1.010684,0.831268,0.727155,0.657699,0.616287", \  
                "2.170611,1.965158,1.760932,1.459438,1.140559,0.930355,0.781393", \  
                "3.276307,3.084566,2.831754,2.426623,1.913607,1.439055,1.113950");  
    }  
}
```



Cell Propagation Delay & Output Transition Time is function of Input Transition and Output Capacitance.



Switching or Internal power is function of Input Transition and Output Capacitance

Libraries Definition

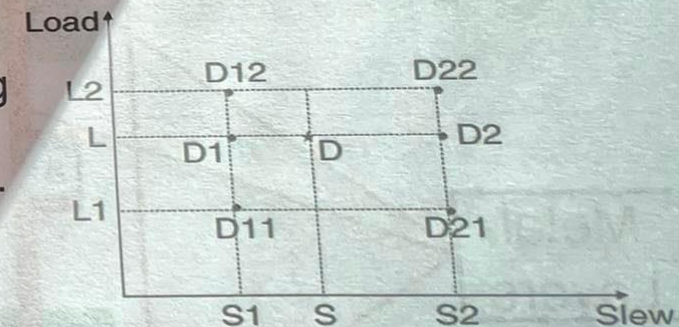
.LIB File

- **Delay Calculation Model: -**

- Synopsys supports several delay models. These include the CMOS generic delay model, CMOS piecewise linear delay model and the CMOS non-linear table lookup model. Presently, the first two models are not in common use, due to their inefficiencies. **The non-linear delay model (NLDM) is the most delay model used in the ASIC world.**

- **The non-linear delay model (NLDM) uses a circuit simulator to characterize a cell's transistors with a variety of input transitions, and output load capacitances and record the results in a table, with corresponding input transition and output load capacitance.**

- If the delay (or, output slew) has to be determined corresponding to load L, and input slew S which not recorded in the table, then in this case, the value can be determined through **Interpolation**.

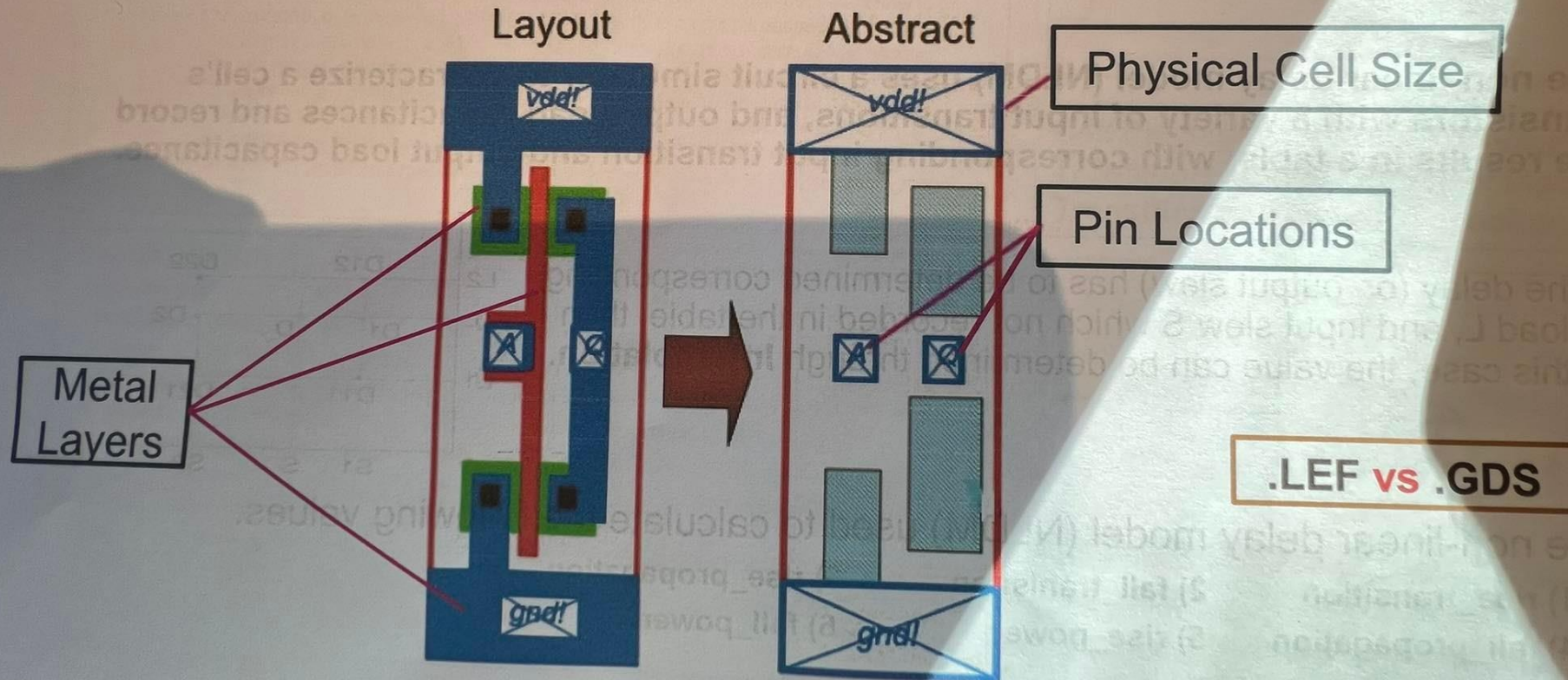


- The non-linear delay model (NLDM) used to calculate the following values.

- | | | |
|---------------------|-------------------|---------------------|
| 1) rise_transition | 2) fall_transiton | 3) rise_propagation |
| 4) fall_propagation | 5) rise_power | 6) fall_power |

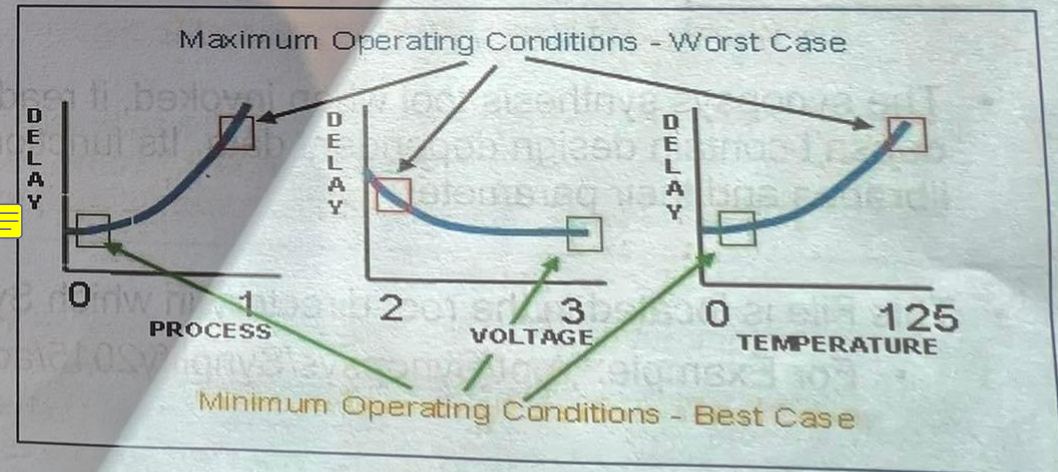
Libraries Definition

- **Physical views**
 1. **Abstract of cells (std_cell.lef) for PnR and RC extraction.**
It contains physical outline of cells as size, shape and pin locations
 2. **Layout of cells (std_cell.gds) for DRC, LVS and custom layout.**
It contains info regarding the layout of the cells and metal layers used.



Operating Conditions

- Standard cells is characterized in the technology library at specific (PVT) **Process** , **Voltage** and **Temperature** which is known as “**operating condition**”.
- In real scenarios we need to consider during synthesis & timing analysis the process, voltage and temp variation across the chip which direct affect **cell delays**. Synthesis tool must consider the **worst-case** and **best-case** delays for the **expected variations** in the temperature, voltage, and process factors.
 - Temperature** – Temperature variation on a chip is a fact of life and can't be avoided. The threshold voltage is a function of temperature in direct proportional relation, so as **temperature increase**, threshold voltage will increase, then **propagation delay increase**.
 - Voltage** – The design's supply voltage can vary from the ideal value during day-to-day operation and it is normal. MosFet current is function of supply in direct proportional relation, so as **voltage decrease**, current will decrease, then **propagation delay increase**.
 - Process** – This variation accounts for deviations in the semiconductor fabrication process parameters which can be **doping**, **impurity densities**, **oxide thicknesses** and **diffusion depths** which introduces variations in the transistor parameters such as threshold voltage, transistor dimensions (W/L). The ICs are produced in approximately 100 dice per wafer. The electrical properties to two adjacent different dice can be very different as well as the electrical properties of different transistors across the same chip can be different.
- Generally there are different libraries specific to different operating condition (also called as **Timing Corner**). There are three operating conditions very commonly used in ASIC synthesis.
 - Worst** (also called 'max' or 'slow') : a library in which cells are characterized for **maximum** delay
 - Best** (also called 'min' or 'fast') : a library in which cells are characterized for **minimum** delay
 - Nominal** (also called 'typical' or 'normal') : a library in which cells are characterized for **typical** delay



Invoking Design Compiler

1. Interactive dc shell :

- `dc_shell -f scriptFile`
- Most efficient and common usage is to put TCL commands into scriptFile.
- Edit and rerun scriptFile as needed

2. GUI version (Design Vision)

- `design_vision`
- From dc_shell: `gui_start`
- Main advantage over dc_shell is to view the synthesized schematic

Startup File

- The synopsys synthesis tool when invoked, it reads a startup file (synopsys_dc.setup). This file doesn't contain design dependent data. Its function is to load the synopsys technology independent libraries and their parameter
- This File is located in the root directory in which Synopsys synthesis tool is installed
 - For Example: `/opt/Synopsys/Synplify2015/admin/setup`

Libraries Definition

• Loading of libraries through Synopsys Design Compiler Tool:-

- Design Compiler has some variables used to specify libraries used during **synthesis, mapping & optimization**.

1. **search_path** variable

- Design Compiler uses **search_library** variable to specify the paths in order to search for design and library files.
- SDC Commands : **lappend search_path "/class/ELEC6250/cmoshvt/std_cell"**

2. **target_library** variable

- Design Compiler uses **target_library** variable to specify only the standard cells **timing libraries** in **db** format which used in mapping the combinational and sequential generic cells to technology equivalent cells during synthesis.
- SDC Command : **set target_library [list CMOSHVT.db CMOSLVT.db]**

3. **link_library** variable

- Design Compiler uses **link_library** to specify the list of libraries (standard cells **timing** libraries, hard macros libraries as Memories, PLL, ...) in **db** format and design files needed in the linking step. For a design to be complete, it must connect all the library components and designs it references. This process is called **linking the designs**
- SDC Command : **set link_library [list "*" CMOSHVT.db CMOSLVT.db RAM64X64.db]**

4. **synthetic_library** variable

- Specifies Synopsys DesignWare components as adders and multipliers for built in HDL operators (+, -, * and so on), Design compiler automatically load this library.
- SDC Command : **set synthetic_library [list standard.sldb]**

Notes: -

1. The asterisk "*" in the value of **link_library** variable specifies that Design Compiler should search for the references in all the design files and libraries loaded in the memory.
2. **.db** format is a compiled version of **.lib (liberty)** file. the compilation is made by **lc_shell** (library compiler, a tool from synopsys) as well, the **.db** & **.lib** files contain the same timing/power/area/ information.

Reading Design Files

- The first task for the synthesis tool is to read the designs and check for any unsynthesized syntax. There are two methods to read files into Design Compiler. One is to use **analyze/elaborate** commands; the other is to use **read_file** command.

1. analyze/elaborate Method:-

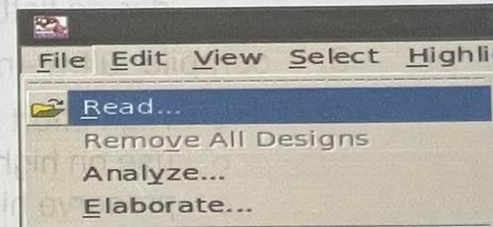
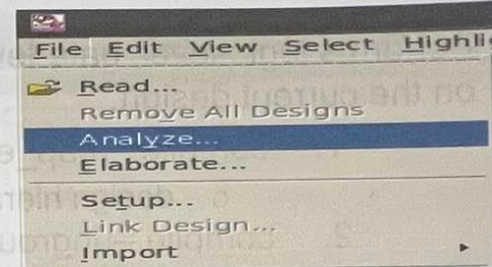
- **analyze** command:-
 - Reads an HDL source file and performs HDL syntax checking and report errors.
 - Creates HDL library objects in an intermediate format (.syn) and store it in work directory.
- **elaborate** command:-
 - Synthesize the design into a technology independent representation "GTECH" from the intermediate format (.syn) generated from analyze command.
 - Allows changing of parameter values defined in the source code.
 - Replaces the HDL arithmetic operators in the code with DesignWare components.
 - **Performs link automatically**

2. read_file Method:-

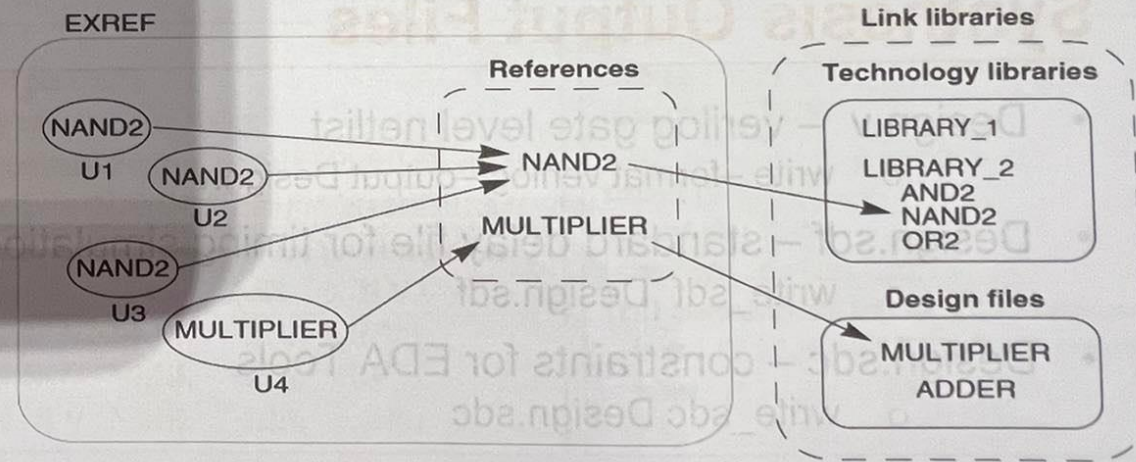
- **read_file** command:-
 - Reads an HDL source file and performs HDL syntax checking and report errors.
 - Translates the design into a technology independent representation "GTECH"
 - Replaces the HDL arithmetic operators in the code with DesignWare components.
 - Cannot pass parameters while reading HDL files
 - Doesn't Create any intermediate files for designs.
 - **Doesn't execute link command automatically**

Reading Design Files

- **1st Method:-**
 - SDC Commands:-
 - analyze -format verilog {ALU.v RegFile.v TOP.v}
 - elaborate -lib work TOP
- **2nd Method:-**
 - SDC Commands:-
 - read_file -format verilog {ALU.v RegFile.v TOP.v}
 - Link



• Linking Designs



Compile

- Performs logic-level, gate-level synthesis and optimization as well as running Static Timing Analysis on the current design. 
 1. `compile -map_effort medium` 
 - design hierarchy preserved
 2. `compile -ungroup_all map_effort high` 
 - design flattened
 3. `compile_ultra -no_auto_ungroup`
 - High effort compile
 - use on high-performance designs and tight time constraints
 - preserve hierarchy

Synthesis Output Files

- Design.v – verilog gate level netlist
 - `write -format verilog -output Design.v`
- Design.sdf – standard delay file for timing simulation 
 - `write_sdf Design.sdf`
- Design.sdc – constraints for EDA Tools 
 - `write_sdc Design.sdc`

Different Arcs From Tech Library

Setup Time

```
pin (D) {
  direction : input;
  ...
  timing () {
    related_pin : "CK";
    timing_type : "setup_rising";
    rise_constraint ("setuphold_template_3x3") {
      index_1 ("0.4, 0.57, 0.84"); /* Data transition */
      index_2 ("0.4, 0.57, 0.84"); /* Clock transition */
      values ( /* 0.4 0.57 0.84 */ \
        /* 0.4 */ "0.063, 0.093, 0.112", \
        /* 0.57 */ "0.526, 0.644, 0.824", \
        /* 0.84 */ "0.720, 0.839, 0.930");
    }
  }
}
```

Hold Time

```
timing () {
  related_pin : "CK";
  timing_type : "hold_rising";
  rise_constraint ("setuphold_template_3x3") {
    index_1 ("0.4, 0.57, 0.84"); /* Data transition */
    index_2 ("0.4, 0.57, 0.84"); /* Clock transition */
    values ( /* 0.4 0.57 0.84 */ \
      /* 0.4 */ "-0.220, -0.339, -0.584", \
      /* 0.57 */ "-0.247, -0.381, -0.729", \
      /* 0.84 */ "-0.398, -0.516, -0.864");
  }
}
```

Clock-To-Q Time

```
timing () {
  related_pin : "CKN";
  timing_type : falling_edge;
  timing_sense : non_unate;
  cell_rise (delay_template_3x3) {
    index_1 ("0.1, 0.3, 0.7"); /* Clock transition */
    index_2 ("0.16, 0.35, 1.43"); /* Output capacitance */
    values ( /* 0.16 0.35 1.43 */ \
      /* 0.1 */ "0.0513, 0.1537, 0.5280", \
      /* 0.3 */ "0.1018, 0.2327, 0.6476", \
      /* 0.7 */ "0.1334, 0.2973, 0.7252");
  }
}
```

Removal Time

```
timing () {
  related_pin : "CP";
  timing_type : removal_rising;
  rise_constraint (removal_template_3x3) { /* CDN rising */
    index_1 ("0.032, 0.504, 0.788"); /* Data transition */
    index_2 ("0.032, 0.504, 0.788"); /* Clock transition */
    values ( /* 0.032 0.504 0.788 */ \
      /* 0.032 */ "0.106, 0.167, 0.548", \
      /* 0.504 */ "0.221, 0.381, 0.662", \
      /* 0.788 */ "0.381, 0.456, 0.778");
  }
}
```

Recovery Time

```
timing () {
  related_pin : "CK";
  timing_type : recovery_rising;
  rise_constraint (recovery_template_3x3) { /* CDN rising */
    index_1 ("0.032, 0.504, 0.788"); /* Data transition */
    index_2 ("0.032, 0.504, 0.788"); /* Clock transition */
    values ( /* 0.032 0.504 0.788 */ \
      /* 0.032 */ "-0.198, -0.122, 0.187", \
      /* 0.504 */ "-0.268, -0.157, 0.124", \
      /* 0.788 */ "-0.490, -0.219, -0.069");
  }
}
```

Minimum Pulse Width

```
pin (CDN) {
  direction : input;
  capacitance : 0.002236;
  ...
  timing () {
    related_pin : "CDN";
    timing_type : min_pulse_width;
    fall_constraint (width_template_3x1) { /* low pulse check */
      index_1 ("0.032, 0.504, 0.788"); /* Input transition */
      values ( /* 0.032 0.504 0.788 */ \
        "0.034, 0.060, 0.377");
    }
  }
}
```


Timing Analysis

- Timing, area, and power constraints drive the operation of synthesis and physical implementation tools. These tools synthesize the netlist and perform physical placement and routing with the goal of making the fastest device, using the least area and power. These tools perform trade-offs between **speed**, **area**, **power**, and **runtime** according to the constraints set by the designer. However, a chip must meet the timing constraints to operate at the intended clock rate, so timing is the most important design constraint. There are two kinds of timing analysis:-

1) Dynamic Timing Analysis:-

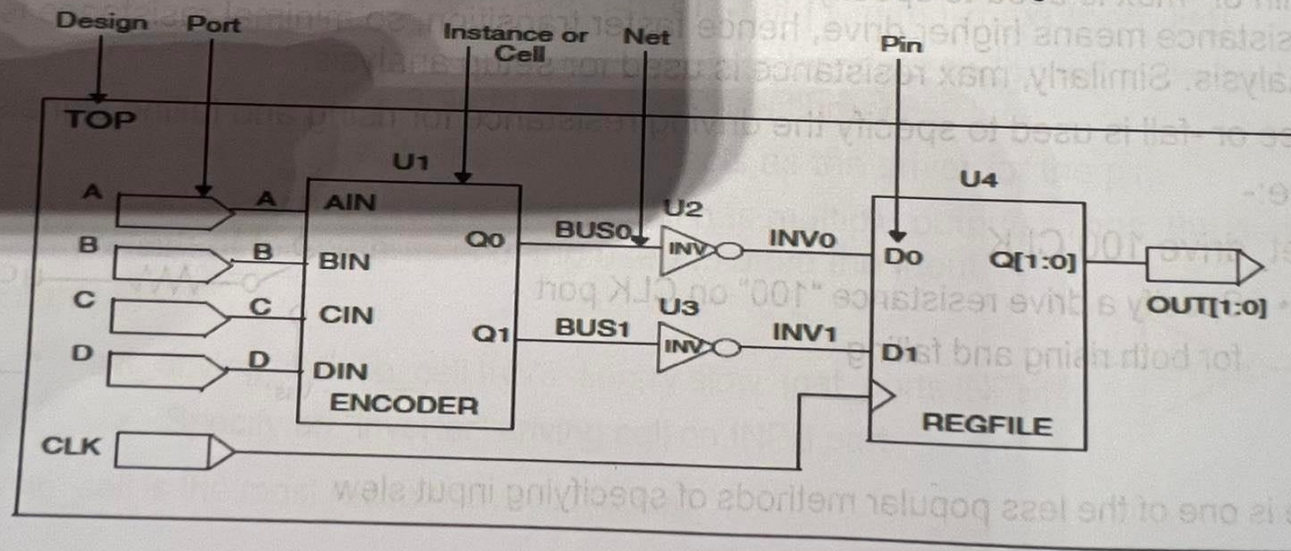
- known as "**simulation based timing analysis**" means we apply a set of vectors at the input signals and observe the time at which the signals reach various points in the circuit. By knowing the time difference between the inputs applied and the signals observed, we know different circuit path delays. And then we know whether or not the different flops meet the setup and the hold requirements.
- Determine the full behavior of the circuit for a given set of input stimulus vectors.
- Slow to run as 100 inputs implies 2^{100} vectors needed to verify timing.
- Check the **functionality as well as the timing** of the circuit.
- Not exhaustive as it is not possible for the whole chip.
- Dynamic timing analysis is **very accurate**.
- Dynamic timing analysis is suitable to validate **both synchronous and asynchronous logic**.

2) Static Timing Analysis:-

- STA is a method of validating the timing performance of a design by checking **all possible paths** for timing violations under worst-case conditions. It considers the worst possible delay through each logic element.
- STA is **static** since the analysis of the design is carried out statically and does not depend upon the data values being applied at the input ports.
- STA checks the design only for proper timing, **not for correct logical functionality**.
- STA provides a **faster** and simpler way of checking and analyzing all timing paths for any timing violations.
- STA is a complete and **exhaustive verification** of all timing checks of a design.
- STA is less accurate as it depends on some models.
- STA is suitable only for synchronous design logic and not suitable for validating asynchronous logic.

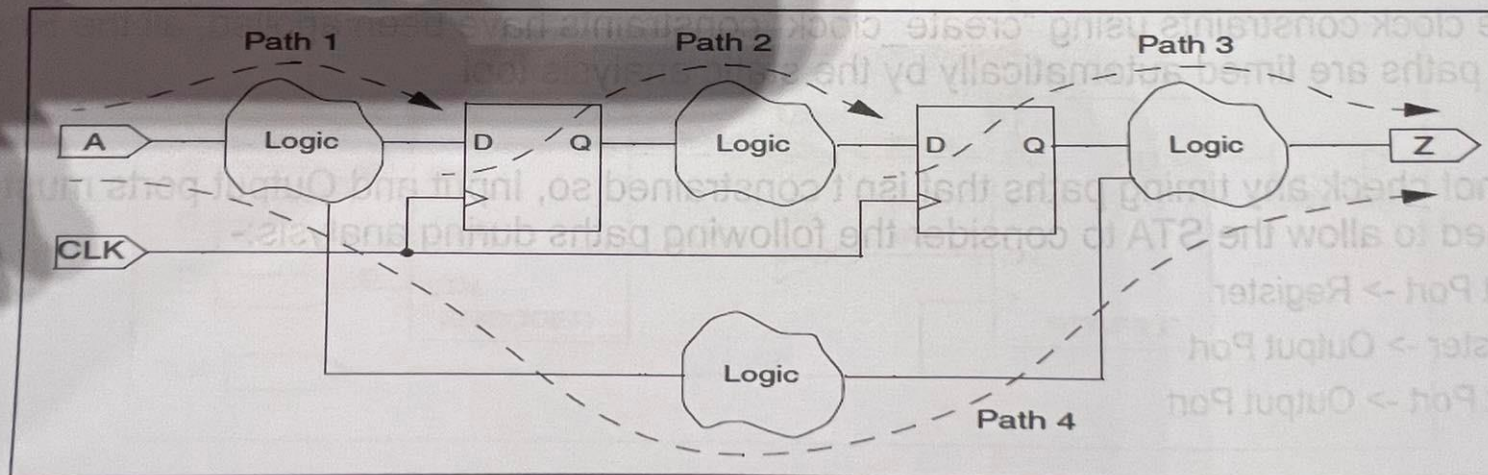
Port | Pin | Net Definition

- **Pin :-**
 - A pin is a point of logical connectivity on a cell. A pin allows the internals of a cell to be abstracted away and simplified for easier use, and can either be on hierarchical or primitive cells. Examples of pins include clock, data, reset, and output pins of a flop.
- **Port :-**
 - A port is a special type of hierarchical pin, a pin on the top level netlist object, module or entity. Ports are normally attached to I/O pads and connect externally to the FPGA device.
- **Net:-**
 - A net is a wire or list of wires that eventually be physically connected directly together. Nets can be hierarchical or flat, but always sorts a list of pins together.



Static Timing Analysis

- The Timing Analysis Tool finds and analyzes all the timing paths in the design. Each path has a **startpoint** and an **endpoint**. Designs have 4 types of timing paths:
1. Input Port -> Register
 - Path 1 starts at an **input port** and ends at the **data input of a sequential element** (input-to-register paths).
 2. Register -> Register
 - Path 2 starts at the **clock pin of a sequential element** and ends at the **data input of a sequential element** (register-to-register paths).
 3. Register -> Output Port
 - Path 3 starts at the **clock pin of a sequential element** and ends at an **output port** (register-to-output paths).
 4. Input Port -> Output Port
 - Path 4 starts at an **input port** and ends at an **output port** (input-to-output paths).



Design Compiler Constraints

- Design Optimization Constraints: -

1. Master Clocks Definition: -

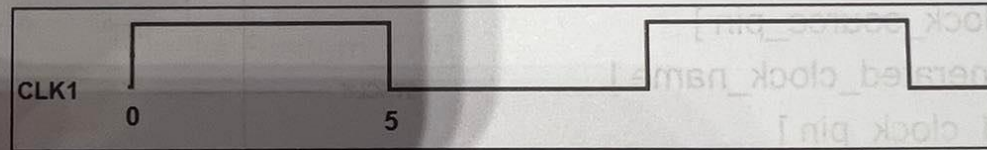
➤ Each clock in the design need to be defined

- SDC Command :-

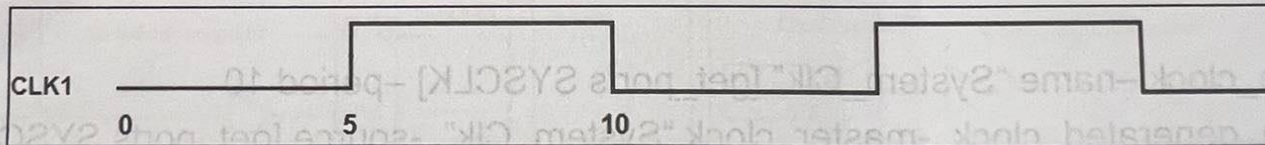
- `create_clock [-name clock_name ]`
 `[ -period clock_period_value ]`
 `[ -waveform {rise_edge fall_edge}]`
 `[ source_port ]`

- Example

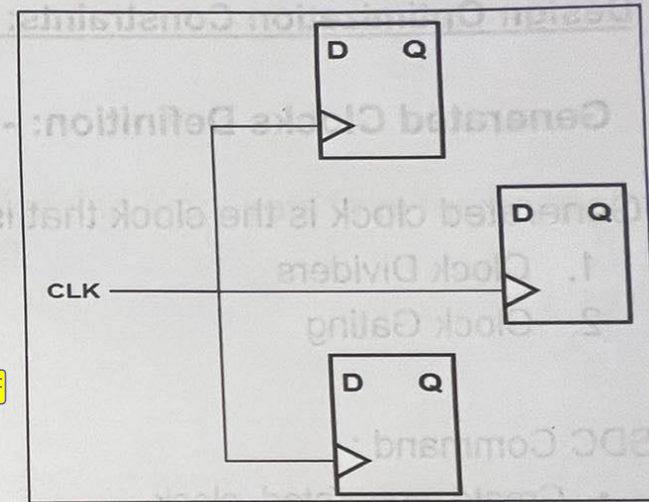
- `create_clock -period 10 -name CLK1 -waveform {0 5} [get_ports CLK]`



- `create_clock -period 10 -name CLK1 -waveform {5 10} [get_ports CLK]`



- Note: When `-waveform` option is not specified, the clock is assumed to have a 50 % duty cycle. This is equivalent to saying that the waveform is `{ 0 period/2 }`.



Design Compiler Constraints

- Design Optimization Constraints: -

2. Generated Clocks Definition: -

- Generated clock is the clock that is generated by on-chip logic from another clock as :-
 1. Clock Dividers
 2. Clock Gating

- SDC Command :-

- Create_generated_clock

[-master_clock master_clock_name]

[-source clock_source_pin]

[-name generated_clock_name]

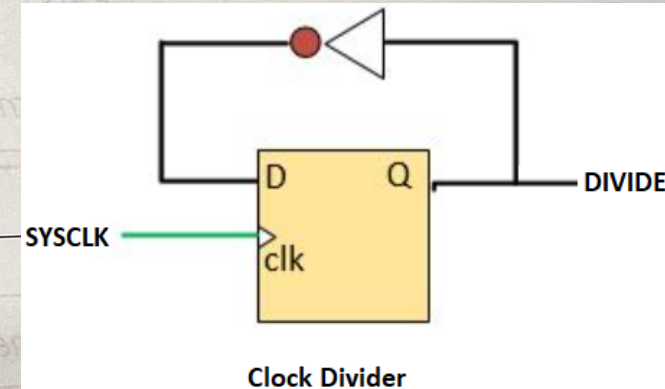
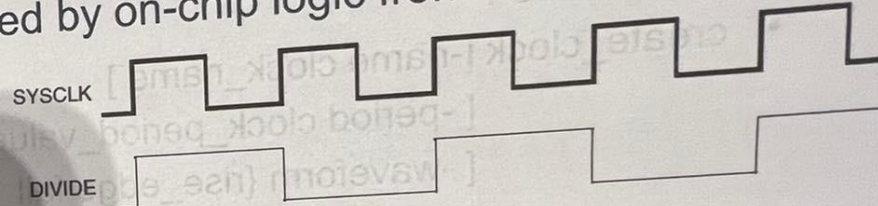
[generated_clock_pin]

[-divide_by factor]

- Example

- create_clock -name "System_Clk" [get_ports SYSCLK] -period 10

- create_generated_clock -master_clock "System_Clk" -source [get_ports SYSCLK] \
- name "DIVIDE" -divide_by 2 [get_pins i_Clock_Divider/FF1/Q]



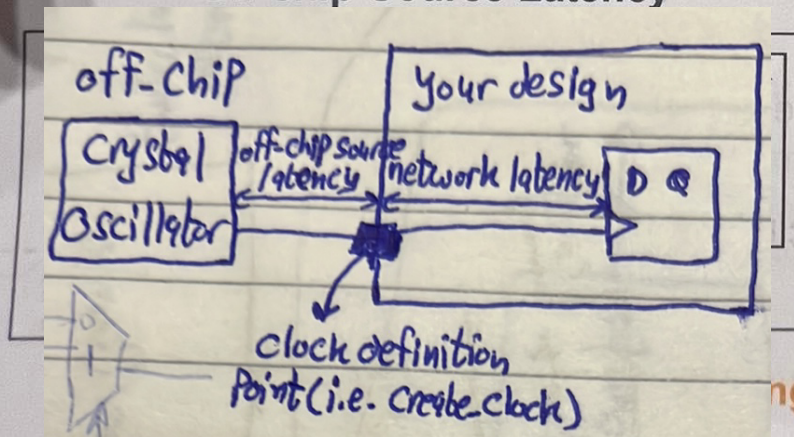
Design Compiler Constraints

- Design Optimization Constraints: -

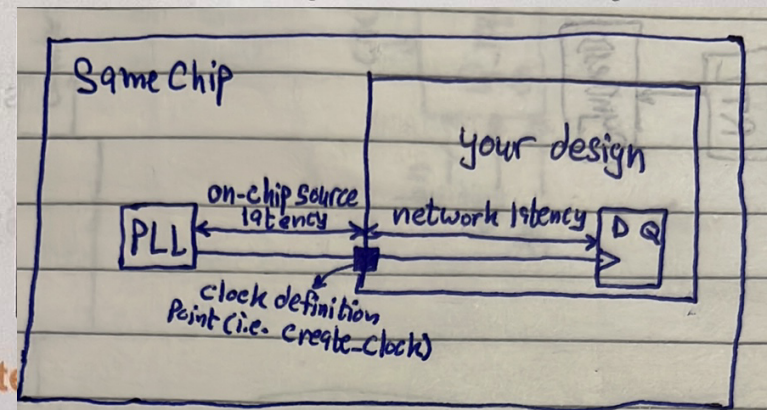
3. Clock Latency: -

- **Clock Latency** is the amount of time it takes for the clock signal to propagate from the original clock source to the clock pin of sequential elements in the design.
- SDC Command :-
 - `set_clock_latency [-min] [-max]`
`delay_value`
`[ source_port ]`
- Example
 - `set_clock_latency 5 [get_clocks System_Clk]`

Off-chip Source Latency



On-chip Source Latency



Design Compiler Constraints

- Design Optimization Constraints: -

4. Clock uncertainty: -

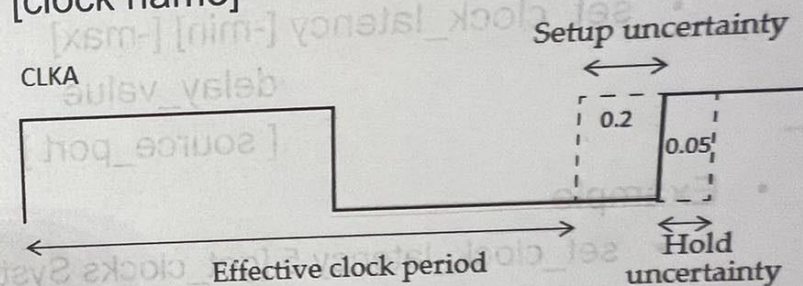
- **Clock Uncertainty** is the time variation in the arrival times of the clock edges resulting from clock jitter and clock skew.

- SDC Command :-

- `set_clock_uncertainty [-setup] [-hold] delay_value [clock name]`

- Example: -

- `set_clock_uncertainty -setup 0.2 [get_clocks CLK_A]`
- `set_clock_uncertainty -hold 0.05 [get_clocks CLK_A]`



5. Clock Transition: -

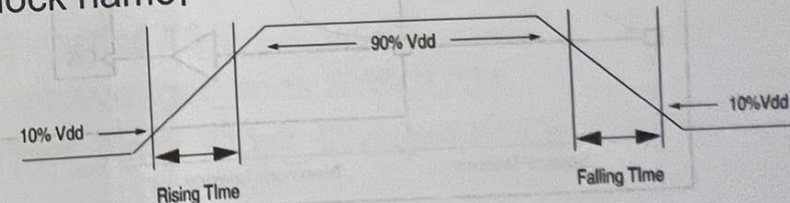
- **Clock Transition** is the rise time and fall time of the clock signal.

- SDC Command :-

- `set_clock_transition [-rise] [-fall] transition_time [clock name]`

- Example: -

- `set_clock_transition 0.1 [get_clocks System_Clk]`



Design Compiler Constraints

-- Drive Characteristics at Input Ports --

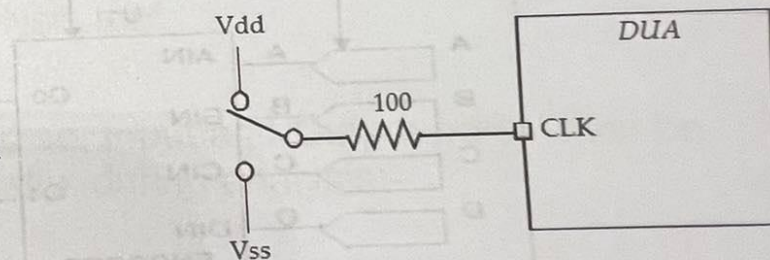
1. **set_drive** command :

- The **set_drive** explicitly specifies a value for the drive resistance at the input port of the design.
- Given the resistive value of the driver and knowing the capacitive value of the input pin, the tools can compute the slew time at the input, which used to calculate the gates delays driven by that port
- The **high resistance** value mean **slower transition** and **low resistance** value mean **faster transition**.
- SDC Command:-

- **set_drive** [-rise] [-fall] [-min] [-max] **resistance_value** {port_list}
- **-min** or **-max** is used to specify the minimal resistance or maximum resistance. Since minimal resistance means higher drive, hence faster transition, so minimal resistance is used for hold analysis. Similarly, max resistance is used for setup analysis.
- **-rise** or **-fall** is used to specify the driving resistance for rising and falling signals respectively.

• Example:-

- **set_drive 100 CLK**
 - Specify a drive resistance "100" on CLK port for both rising and falling



- Note: **set_drive** is one of the less popular methods of specifying input slew

Design Compiler Constraints

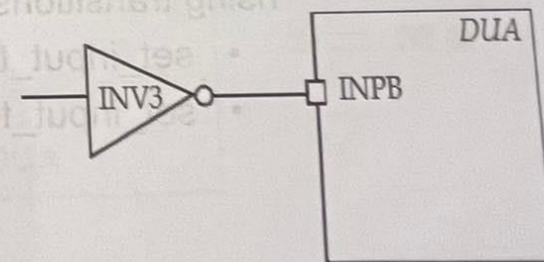
-- Drive Characteristics at Input Ports --

2. **set_driving_cell** command

- The **set_driving_cell** specify a cell library driving the input port.
- Most of the times, the internals of a cell are known only to the circuit designers for the ASIC library, rather than to the people who are using the libraries to create their designs. So, it is much easier to specify the cell which will drive the input, rather than knowing the actual resistive value. The timing analysis tools have enough information from the library about the cell's electrical characteristics that they will be able to extract the relevant information.
- SDC Command:-
 - **set_driving_cell** [-library lib_name] [-lib_cell lib_cell_name] [-pin pin_name] [-rise] [-fall] [-min] [-max] {port_list}
 - **-library** is needed sometimes as multiple libraries might be loaded in the tool. And there might be cells with the same name in more than one library.
 - **-lib_cell** is used to specify the cell which acts as the driver for the pin.
 - **-pin** is used if you are using a driving cell has multiple outputs. Thus, the designer should clarify which of the outputs is being used to drive the input.
 - Example:-

- **set_driving_cell -lib_cell INV3 -library slow [get_ports INPB]**
 - Specify an "inverter" driving cell on INPB port

- Note: **set_driving_cell** is the most recommended method



Design Compiler Constraints

-- Drive Characteristics at Input Ports --

3. **set_input_transition** command :

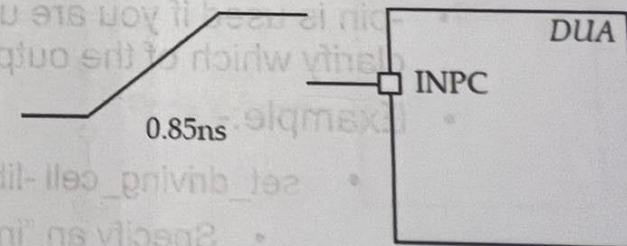
- The commands **set_drive** and **set_driving_cell** are used by the tools to compute the transition time at the input port. However, a designer could specify a fixed transition time directly using **set_input_transition** command.

- SDC Command:-

- **set_input_transition** [-rise] [-fall] [-min] [-max] **transition_value** {port_list}
- -min or -max is used to specify the **minimal** transition time to be used for hold analysis or **maximum** transition time to be used for setup analysis.
- -rise or -fall is used to specify the rise transition time or fall transition time respectively.

- Example:-

- Specifies an input transition of 850ps on port INPC for both rising and falling transitions
 - **set_input_transition 0.85 [get_ports INPC]**
- Specifies an input transition of 850ps on all ports for rising transitions and 350ps for falling transitions.
 - **set_input_transition -rise 0.85 [all_inputs]**
 - **set_input_transition -fall 0.35 [all_inputs]**

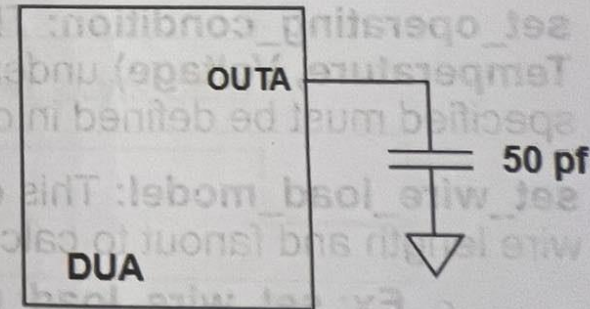


Design Compiler Constraints

-- Port Load Capacitance --

4. **set_load** command :

- To accurately perform timing analysis, the tool needs information about the **external load capacitance** of nets connected to the top-level output ports, including pin capacitance and wire capacitance.
- The **set_load** command places a capacitive load on output ports to model the external load being driven by the output port. By default, the capacitive load on ports is **0**.
- SDC Command:-
 - `set_load [-pin_load] [-wire_load] [-min] [-max] cap_value {port_list}`
 - **-min** or **-max** is used to specify min & max loads during hold analysis and setup analysis.
- Example:-
 - Specifies an input transition of 50ps on port INPC for both rising and falling transitions
 - `set_load 50 [get_ports OUTA]`



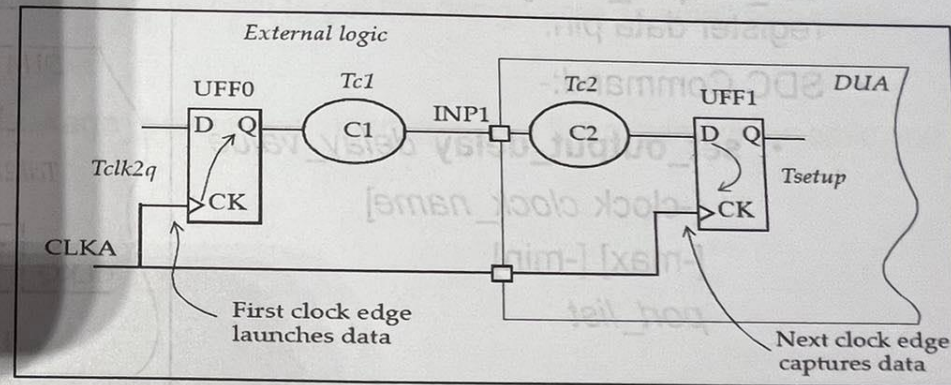
Design Compiler Constraints

• Design Optimization Constraints: -

6. Constraining Input Ports (Input Paths): -

- For each input port, we need to specify the time at which the signal arrives at input port. It is known as "Input Arrival Time" or "Input Delay".
- **set_input_delay** command is used to set the **input delay (input arrival time)** on an input port relative to a clock signal edge
- SDC Command:-

- **set_input_delay** *delay_value*
[-clock *clock_name*]
[-max] [-min]
port_list



- Ex : `set_input_delay 2 -clock CLKA [get_ports INP1]`
- The constraint specifies that the external delay on input INP1 is 2ns and this is with respect to clock CLKA. Assuming the clock period for CLKA is 3ns, then the logic for INP1 pin has only 1ns (= 3ns - 2ns) available for propagating internally in the design

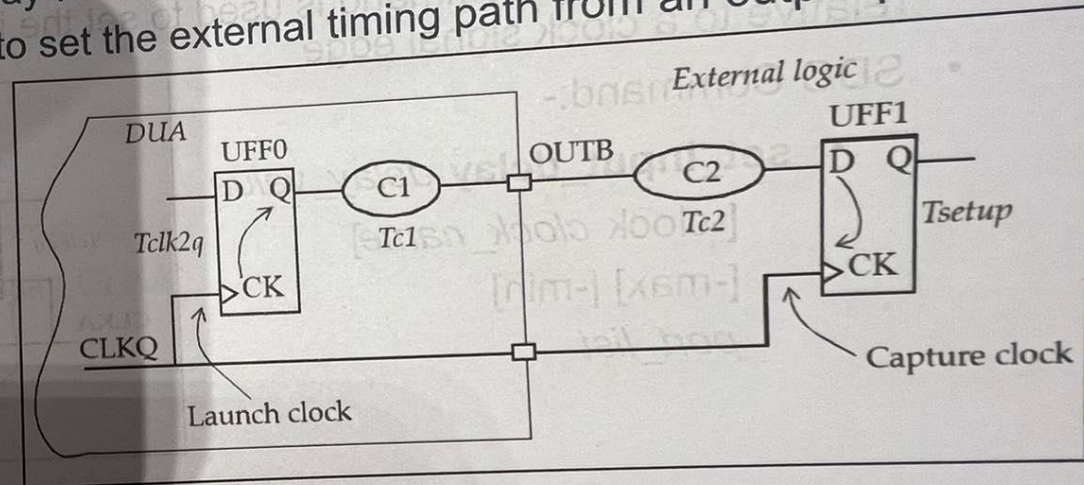
Design Compiler Constraints

- Design Optimization Constraints: -

7.-Constraining Output Ports (Output Paths): -

- For each output port, we need to specify the time that the signal needs to travel after the output port. It is known as "Output Delay".
- **set_output_delay** command is used to set the external timing path from an output port to a register data pin.
- SDC Command:-

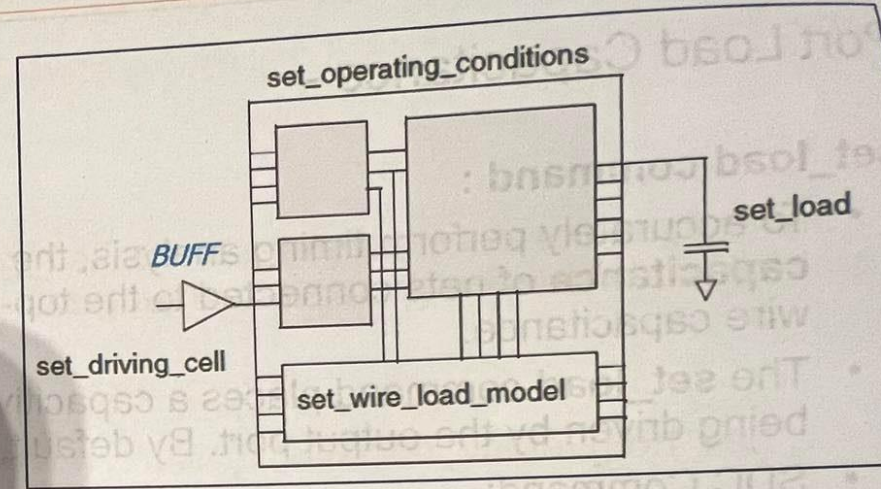
- **set_output_delay** *delay_value*
[-clock *clock_name*]
[-max] [-min]
port_list



- Ex : set_output_delay 2 -clock CLKQ [get_ports OUTB]
- The constraint specifies that the external delay from output "OUTB" is 2ns and this is with respect to clock CLKQ. Assuming the clock period for CLKQ is 4ns, then the logic for "OUTB" port has only 2ns (= 4ns - 2ns) available for propagating externally, so Tc1 plus Tclk2q of UFF0 must be less than 2ns for the flip-flop UFF0 to reliably capture the data by flip-flop UFF1.

Design Compiler Constraints

- Design Environment Constraints: -



1. **set_driving_cell:** Use to set the transition characteristics of the top-level input ports.
 - Ex: **set_driving_cell -lib_cell BUFF -library slow [get_ports IN1]**
2. **set_load:** Used to set the external load capacitance connected to the top-level output ports.
 - Ex: **set_load 15 [get_ports OUT1]**
3. **set_operating_condition:** This command defines the operating conditions (**Process, Temperature, Voltage**) under which to optimize the current design. The operating conditions specified must be defined in one of the loaded std cell libraries.
4. **set_wire_load_model:** This command defines a wire load model used to estimate the effect of wire length and fanout to calculate wire capacitance, resistance to find out wire delays.
 - Ex: **set_wire_load_model "10X10"**

Design Compiler Constraints

- Design Optimization Constraints: -

8. Constraining Design Area: -

- **set_max_area** command specifies the maximum area of the synthesized design
- SDC Command:-
 - **set_max_area 0**
 - Specifying 0 area results in the tool to try its best to get the design area as small as possible.

9. Constraining Design Power: -

- **set_max_dynamic_power** & **set_max_leakage_power** command specifies the maximum dynamic and leakage power of the synthesized design
- SDC Command:-
 - **set_max_dynamic_power 8 mW**
 - **set_max_leakage_power 20 uW**
 - Specifying 0 leakage & dynamic power results in the tool to try its best to get the design power small as possible

10. DC Optimization and buffering: -

- **set_dont_touch_network** command is used to prevent DC from buffering nets or ports, this command is very important for clocks and resets network.
- SDC Command:-
 - **set_dont_touch_network {CLK RST}**

Reporting

• Area Report

- The **report_area** command lists the area of the loaded designs. The report includes the following information:-

- Number of combinational cells
- Number of sequential
- Combinational area
- Number of nets
- Number of ports
- Noncombinational
- Net Interconnect area (based on the used wireload model)

- SDC Command: **report_area**

```
*****
Report : area
Design : TEST_TOP
Version: E-2010.12
Date   : Sun Oct 24 02:08:37 2010
*****

Library(s) Used:
test_lib (File: test_lib.db)

Number of ports:      565
Number of nets:      9654
Number of cells:      8120
Number of combinational cells: 6594
Number of sequential cells: 1526
Number of macros:      0
Number of buf/inv:    1439
Number of references: 163

Combinational area: 562404.432061
Noncombinational area: 6720840.351067
Net Interconnect area: undefined (Wire load has zero net area)

Total cell area: 7283244.783128
Total area: undefined
```

• Power Report

- The **report_power** command reports the static and dynamic power of the loaded designs. The report includes the following information:-

- Switching power
- Internal Power
- Leakage Power
- Total Power

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.0000	0.0000	0.0000	0.0000	(0.00%)	
memory	0.0000	0.0000	0.0000	0.0000	(0.00%)	
black_box	0.0000	0.0000	0.0000	0.0000	(0.00%)	
clock_network	0.0000	0.0000	0.0000	0.0000	(0.00%)	
register	8.3700e-03	1.4923e-03	4.7727e+04	9.9100e-03	(2.44%)	
sequential	0.0000	0.0000	0.0000	0.0000	(0.00%)	
combinational	1.1407e-02	0.3838	5.8829e+05	0.3958	(97.56%)	
Total	1.9777e-02 mW	0.3853 mW	6.3601e+05 pW	0.4057 mW		

- SDC Command: **report_power -hierarchy -verbose**

Reporting

- The report_timing provides a report of timing information of the worst design paths for setup checks and hold checks.

- SDC Command for setup analysis: report_timing -nworst 20 -delay_type max
- SDC Command for hold analysis: report_timing -nworst 20 -delay_type min

HOLD Report

Startpoint: Counter_reg[0]
(rising edge-triggered flip-flop clocked by CLK)
Endpoint: Counter_reg[1]
(rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: min

Des/Clust/Port	Wire Load Model	Library
Up_Dn_Counter	tsmc13_wl30	scmetro_tsmc_cl013g
Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
Counter_reg[0]/CK (DFFRHQX8M)	0.00	0.00 r
Counter_reg[0]/Q (DFFRHQX8M)	0.48	0.48 f
U25/Y (BUFX24M)	0.18	0.66 f
U23/Y (BUFX12M)	0.15	0.82 f
U32/Y (NAND2XLM)	0.50	1.31 r
U38/Y (OAI21X2M)	0.36	1.67 f
Counter_reg[1]/D (DFFRHQX8M)	0.00	1.67 f
data arrival time		1.67
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
clock uncertainty	0.39	0.39
Counter_reg[1]/CK (DFFRHQX8M)	0.00	0.39 r
library hold time	-0.06	0.33
data required time		0.33
data required time		0.33
data arrival time		-1.67
slack (MET)		1.34

Setup Report

Startpoint: Down (input port clocked by CLK)
Endpoint: Counter_reg[1]
(rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max

Des/Clust/Port	Wire Load Model	Library
Up_Dn_Counter	tsmc13_wl30	scmetro_tsmc_cl013g
Point	Incr	Path
clock CLK (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	5.00	5.00 r
Down (in)	0.34	5.34 r
U20/Y (CLKNAND2X2M)	1.00	6.33 f
U32/Y (NAND2XLM)	0.89	7.23 r
U38/Y (OAI21X2M)	0.36	7.59 f
Counter_reg[1]/D (DFFRHQX8M)	0.00	7.59 f
data arrival time		7.59
clock CLK (rise edge)	10.00	10.00
clock network delay (ideal)	0.00	10.00
clock uncertainty	-5.00	5.00
Counter_reg[1]/CK (DFFRHQX8M)	0.00	5.00 r
library setup time	-0.17	4.83
data required time		4.83
data required time		4.83
data arrival time		-7.59
slack (VIOLATED)		-2.77

Design Compiler Constraints

• Timing Exceptions Constraints: -

- As per our previous knowledge, most of the paths in a design need to meet (setup/hold) timing requirements, so that the launched signal by the source flop should be captured at the destination flop at the immediately following active edge of the clock (within one clock cycle). However, there are certain paths which need not follow the above requirements. These paths are called Timing exceptions:-

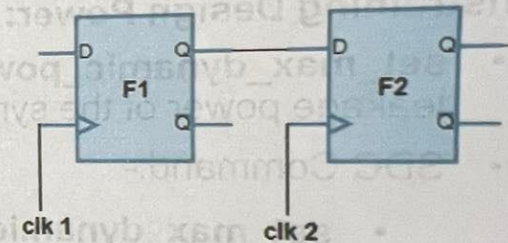
- False Paths:** These are paths which structurally existing or connected but need **not to be timed**. The very dominant reason, why such paths need not be timed is due to CDC (clock domain crossing), there is always some combination of clock edges, which will result in a timing violation. so these paths need to be excluded from timing analysis.

• SDC Command:-

- `set_false_path -hold -setup -from <launching_clock>`
`-to <destination_clock>`

• Examples :

- Disable hold & setup checks for all paths launched by flops triggered by clk1 and captured by flops triggered by clk2 and all paths launched by flops triggered by clk2 and captured by flops triggered by clk1
 - `set_false_path -from [get_clocks clk1] -to [get_clocks clk2]`
 - `set_false_path -from [get_clocks clk2] -to [get_clocks clk1]`
- Disable hold checks for all paths launched by flops triggered by clk1 and captured by flops triggered by clk2 and all paths launched by flops triggered by clk2 and captured by flops triggered by clk1
 - `set_false_path -hold -from [get_clocks clk1] -to [get_clocks clk2]`
 - `set_false_path -hold -from [get_clocks clk2] -to [get_clocks clk1]`



Design Compiler Constraints

• Clock Grouping Constraints: -

- When a design has more than one clock, the timing of such a design depends not just on the frequency of clocks but also on the relation the clocks share with each other. The relationship between two clocks can be:-

• SDC Command:-

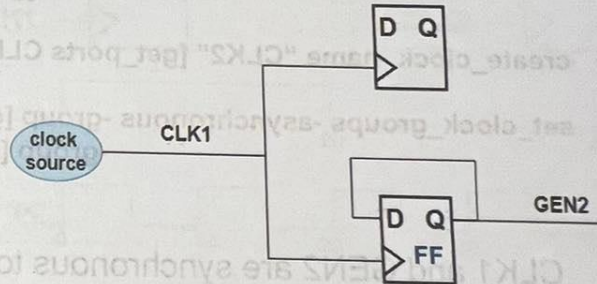
- `set_clock_groups -name <name> -asynchronous -group <clock_list>`

1. **Synchronous** : Two clocks are **synchronous** with respect to each other if they share a common source and have a **fixed phase relationship** as well as the generated clocks.

◦ Examples :-

- `create_clock -name "CLK1" [get_ports CLK1] -period 50`
- `create_generated_clock -master_clock "CLK1" -source "CLK1" -name "GEN2" [get_pin FF/Q] -divide_by`

- The tool assumes that all the defined clocks are synchronous if there is any path with data launched by one clock and captured by the other clock. Unless you specify otherwise so CLK1 and GEN2 are synchronous as they share the same clock phase relationship.

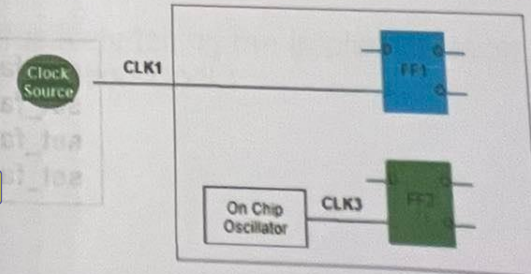


2. **Asynchronous** : Two clocks are **Asynchronous** with respect to each other if they are coming from **different clock sources** as they don't have a fixed phase relationship.

- For asynchronous clocks it is impossible to meet the setup and hold requirements for reliable data capture. We need to indicate to the timing tool to ignore any timing paths

◦ Examples :-

- `create_clock -name "CLK1" [get_ports CLK1] -period 50`
- `create_clock -name "CLK3" [get_ports CLK3] -period 70`
- `set_clock_groups -asynchronous -group "CLK1" -group "CLK3" \`



Design Compiler Constraints

- Clock Grouping Constraints: -

Examples :-

- `create_clock -name "CLK1" [get_ports CLK1] -period 50`
- `create_generated_clock -master_clock "CLK1" -source "CLK1" -name "GEN2" [get_pin FF/Q] -divide_by 2`
- `create_clock -name "CLK2" [get_ports CLK2] -period 70`
- `set_clock_groups -asynchronous -group [get_clocks "CLK1 GEN2"] \ -group [get_clocks CLK2]`

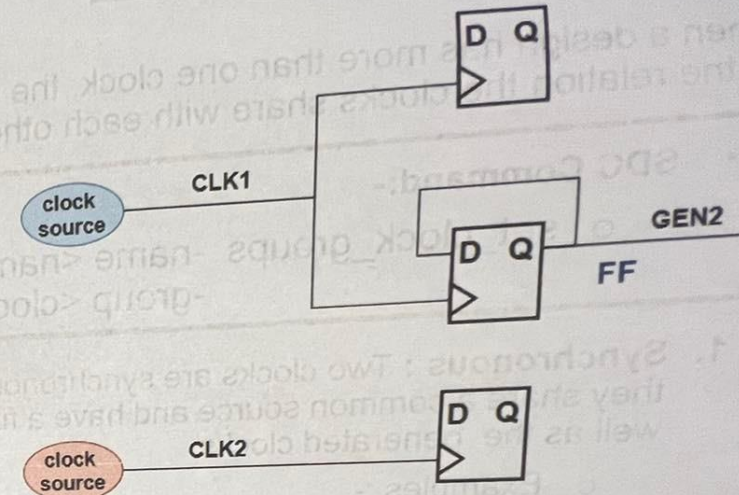
- CLK1 and GEN2 are synchronous to each other, so they are in the same clock group, while CLK2 is asynchronous clock for both CLK1 and GEN2.

- **Note:-**

```
set_clock_groups -asynchronous -group [get_clocks "CLK1 GEN2"] -group [get_clocks CLK2]
```

Is Equivalent To

```
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks CLK1]
set_false_path -from [get_clocks GEN2] -to [get_clocks CLK2]
set_false_path -from [get_clocks CLK2] -to [get_clocks GEN2]
```



Design Compiler Constraints

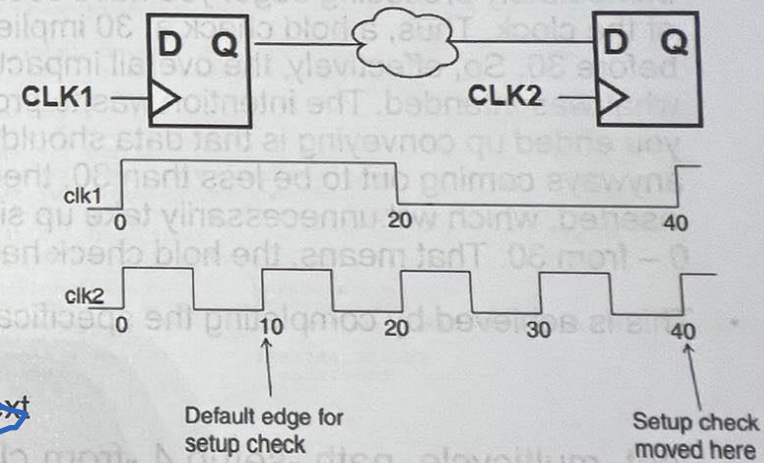
• Timing Exceptions Constraints: -

- 2) **Multicycle Paths:** These are some paths not necessary for a launched data to be captured at the destination in the immediately following clock edge (within one clock cycle). There are some cases, the data maybe allowed to take **more than one clock cycle** to reach the capture flop. Such cases are called Multicycle. So, effectively, a Multicycle Path spec tells STA that the time for capturing the data can be extended beyond the normal single cycle

• SDC Command:-

- `set_multicycle_path -setup <number> -hold <number> -from <clk_name> -to <clk_name>`

- Consider a **Slow to Fast Clock Transfer of Data**, where data is being generated by a slow clock, and is being captured by a fast clock, which is some multiple (in terms of frequency) of the generating clock.
- Assume the flop F1 is clocked by a clock (clk1) with period 40, and F2 is clocked by another clock (clk2) with period 10. as shown Data is launched by F1 at time 0. STA will try to check if the data will reach F2 before the next clock edge on F2, i.e. at 10. But, for next several clocks on F2, there is no new data – which can overwrite the current data. So, F2 might as well capture this new data at 40, or, anytime before it.



`set_multicycle_path -setup 4 -from clk1 -to clk2`

- So, you can declare this path to be a `set_multicycle_path` of 4. This declaration is effectively telling the implementation tools that they can take enough time to have this data reach F2 even up to 4 cycles (of destination clock)

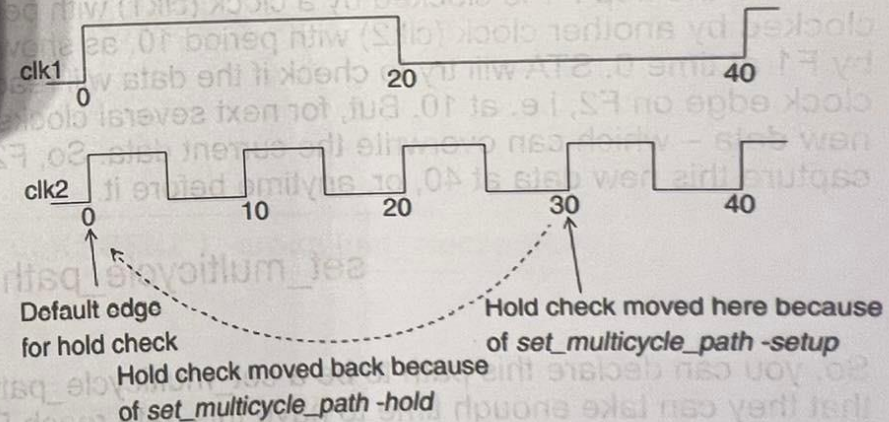
Design Compiler Constraints

• Timing Exceptions Constraints: -

- The hold checks are done with respect to the destination edge immediately preceding the edge used for setup. Thus, moving the setup edge causes the hold edge also to be moved automatically. So, with `set_multicycle_path` being specified as 4, the setup check is being made at 40. That means, the hold check is being made at 30, which is the immediately preceding edge. you have seen that hold check means data should remain stable after the active edge of the clock. Thus, a hold check at 30 implies that the launched data should not reach the destination flop anytime before 30. So, effectively, the overall impact has been: data has to reach only between 30 and 40. But, this is not what was intended. The intention was to provide a flexibility that data can reach anytime till the 4th clock edge. What you ended up conveying is that data should reach ONLY in the 4th cycle. If the actual delay for the path was anyways coming out to be less than 30, then this hold requirement (at 30) will cause additional delay elements to be inserted, which will unnecessarily take up silicon area as well as power. So, you need to move the hold check back to 0 – from 30. That means, the hold check has to be moved back (towards 0 – away from the setup edge) by 3 cycles.
- This is achieved by completing the specification as:

`set_multicycle_path -setup 4 -from clk1 -to clk2`

`set_multicycle_path -hold 3 -from clk1 -to clk2`



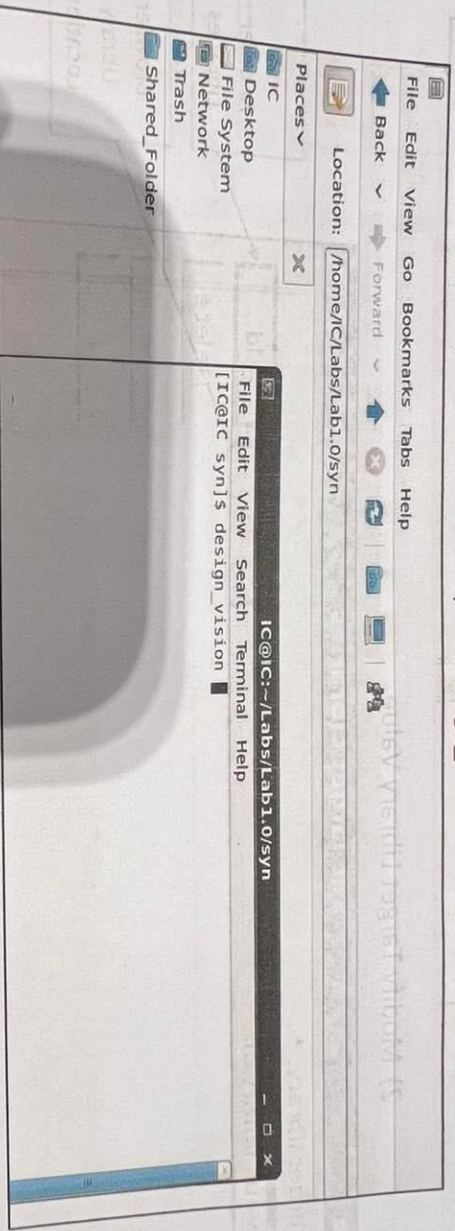
Design Compiler in GUI Mode

Here the steps to use design compiler to:-

- Load technology libraries
- Read RTL design files
- Applying Design Constraints
- Compile (Map & Optimize) the design
- Generate gate level netlist
- Generate reports
- View Design Schematic

Steps:-

1. Invoke design Compiler in GUI Mode by run `design_vision` command in the terminal

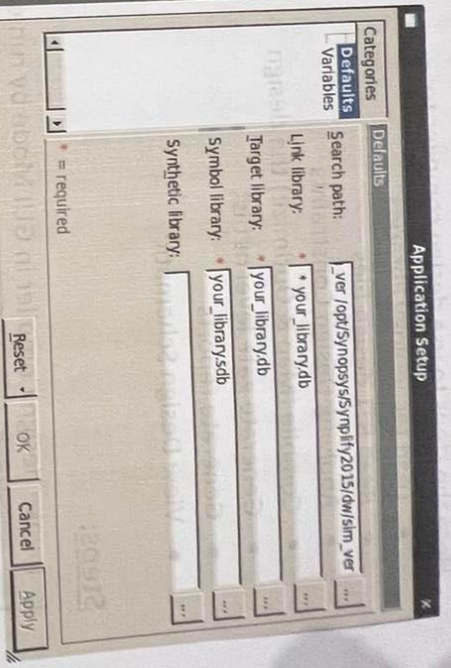
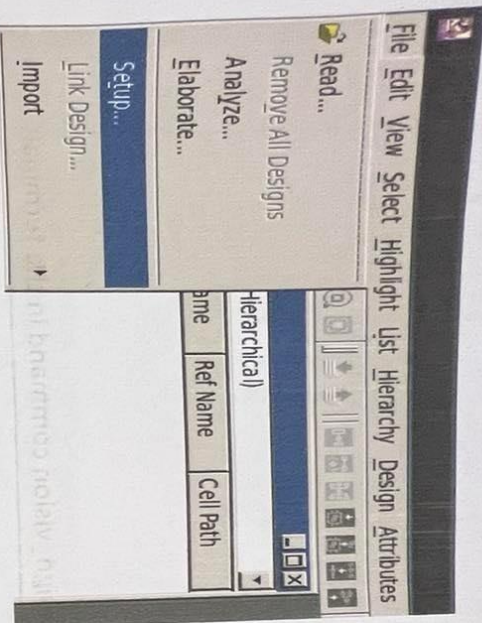


2. Design Compiler GUI will open

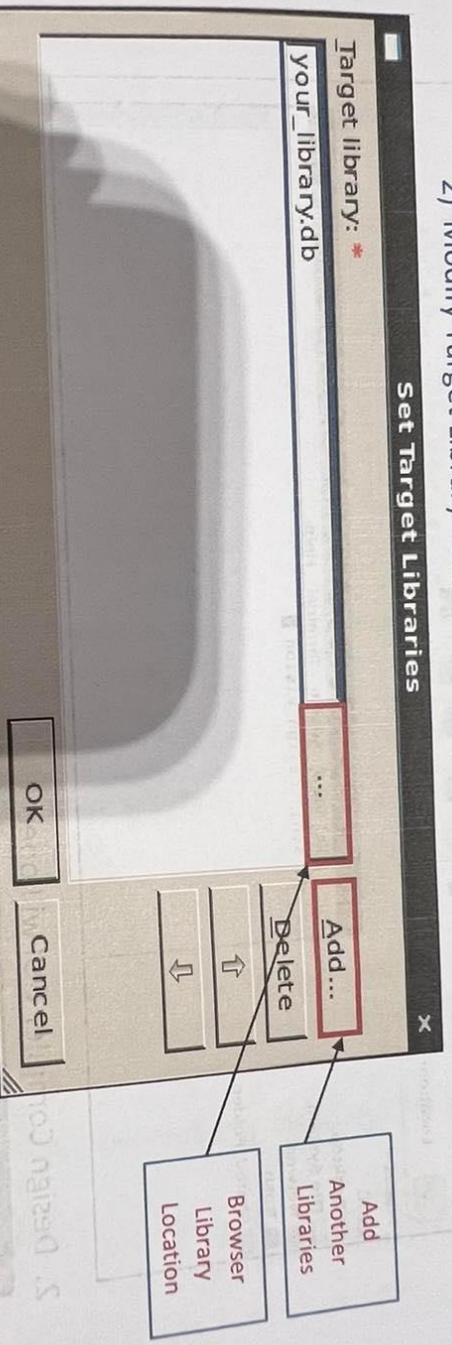


3. Load Technology Libraries

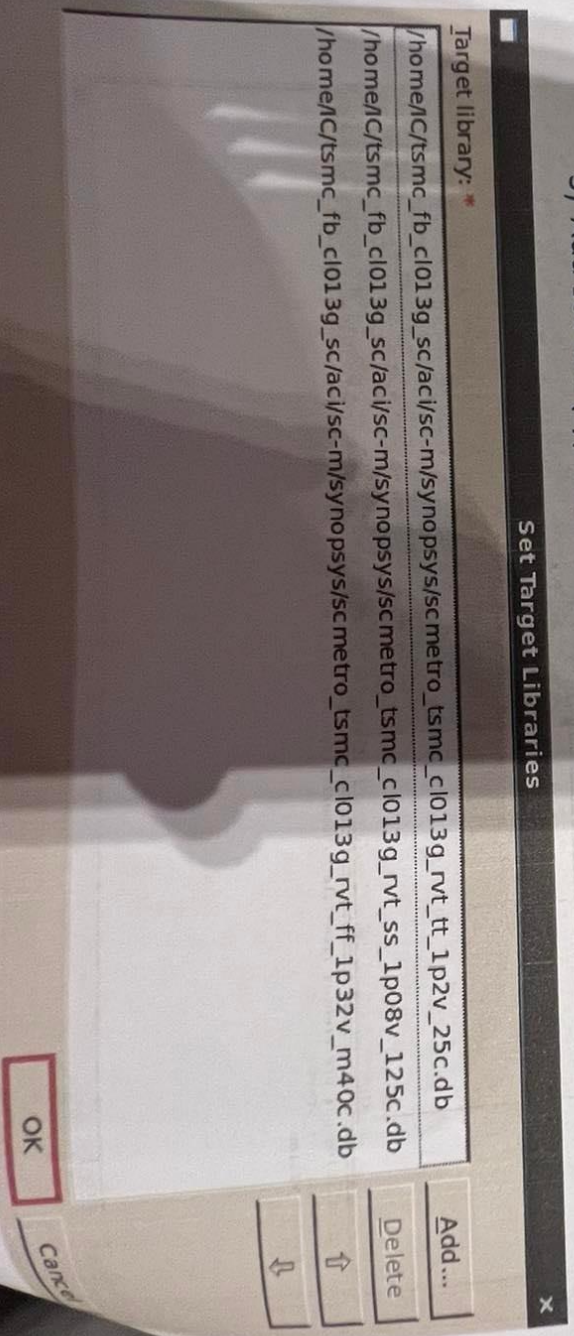
1) Select ... **File** > **Setup**, Libraries Setup window will appear



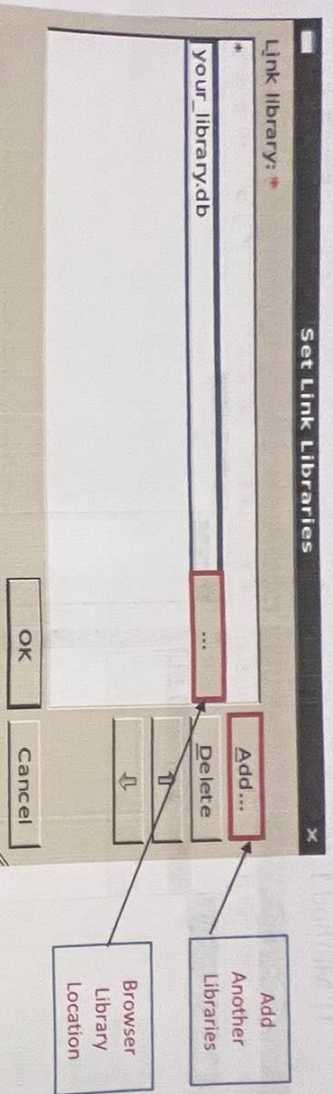
2) Modify Target Library Value



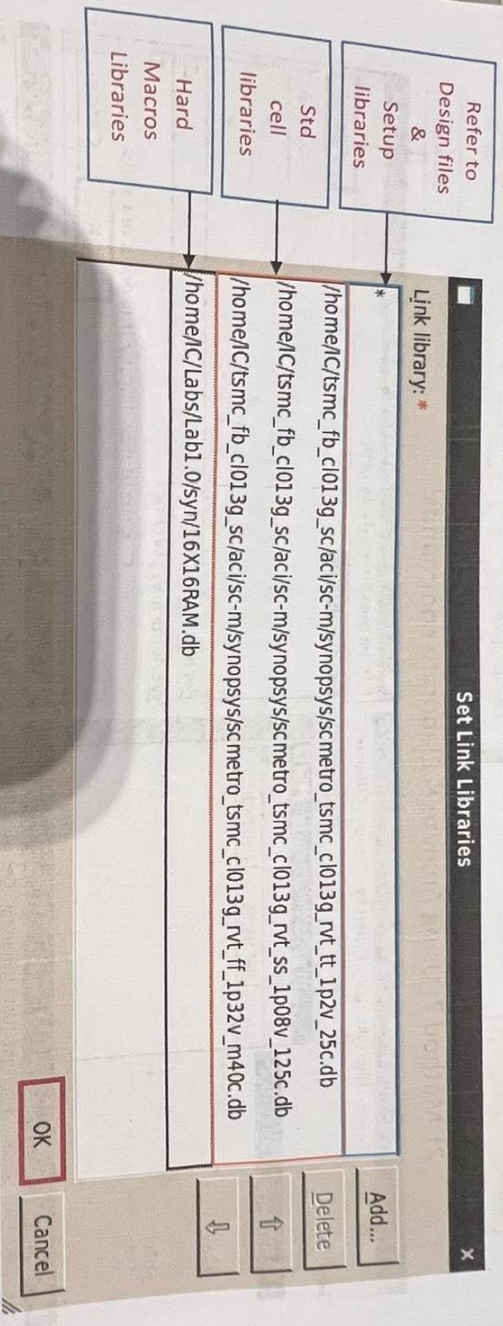
3) Add Std Cell (typical, worst, best) Libraries in db format then click OK



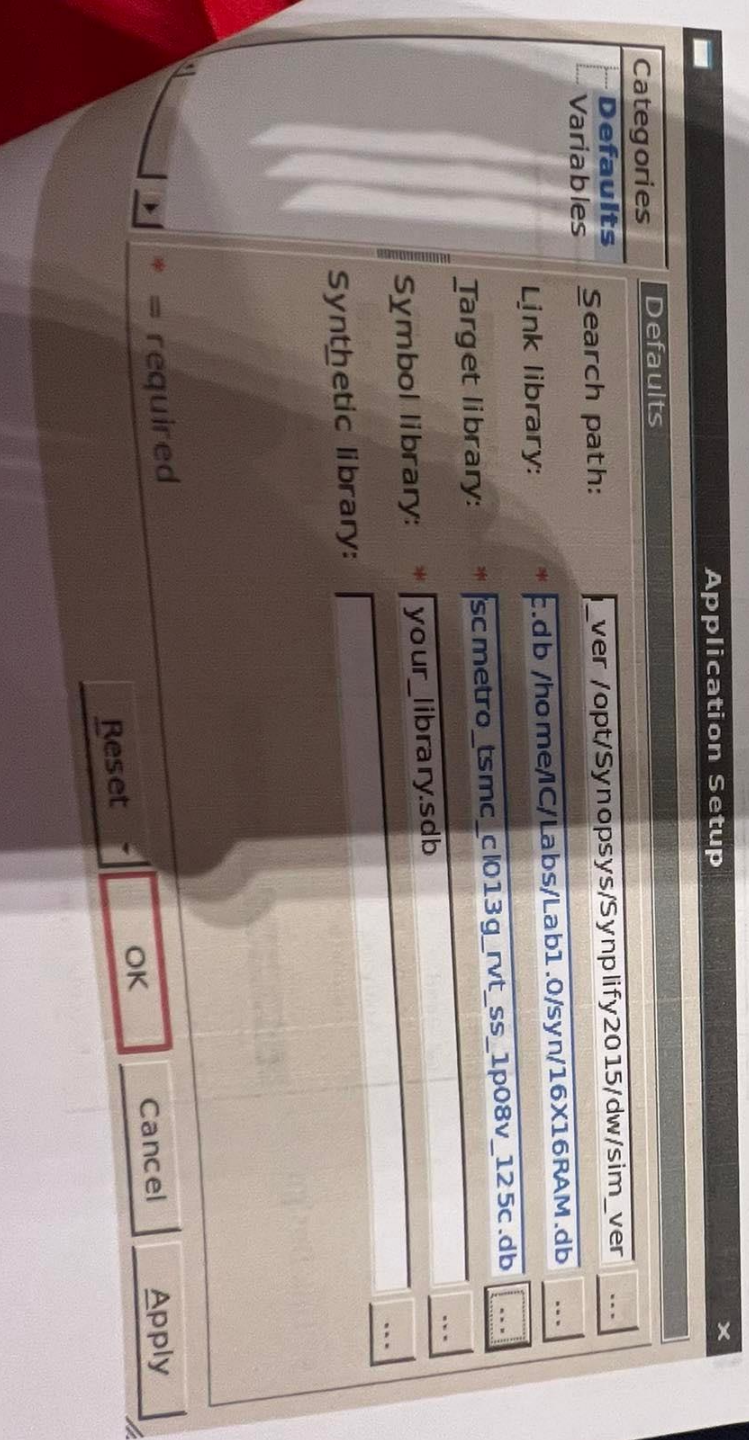
4) Modify Link Library value



5) Add Standard cell and hard macros libraries then click ok

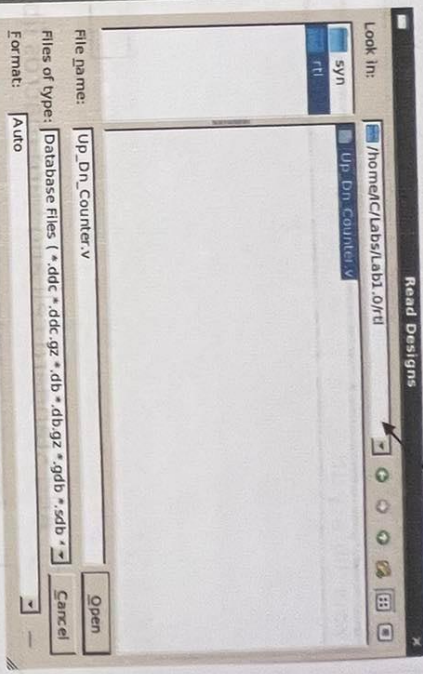
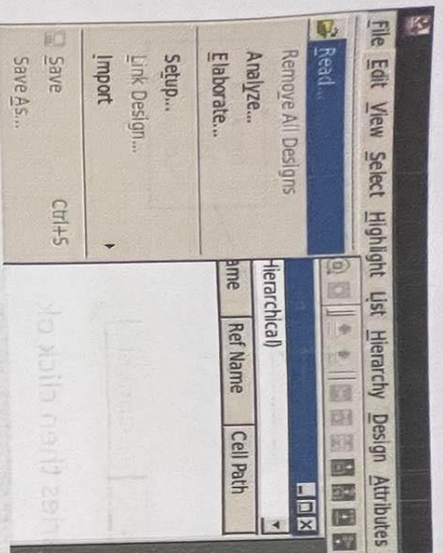


6) After modifying target_library and link_library variables then click OK

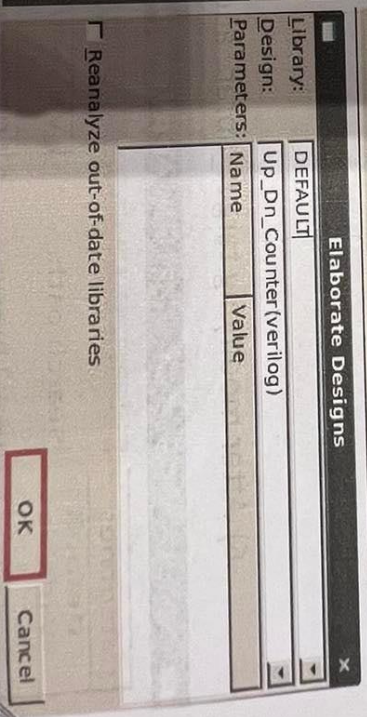
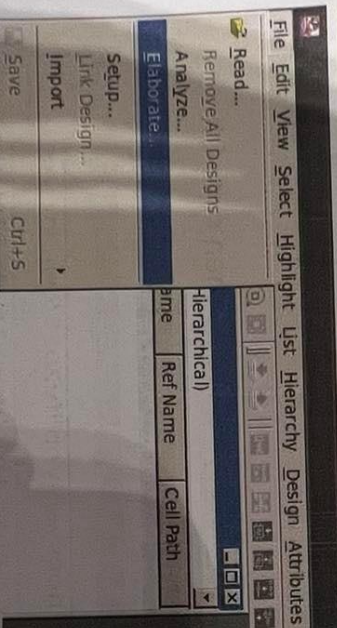
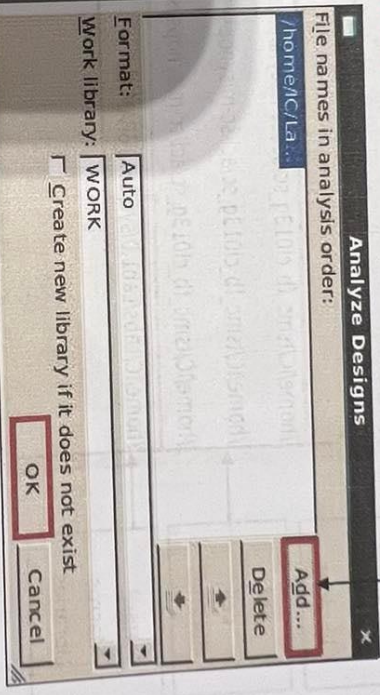
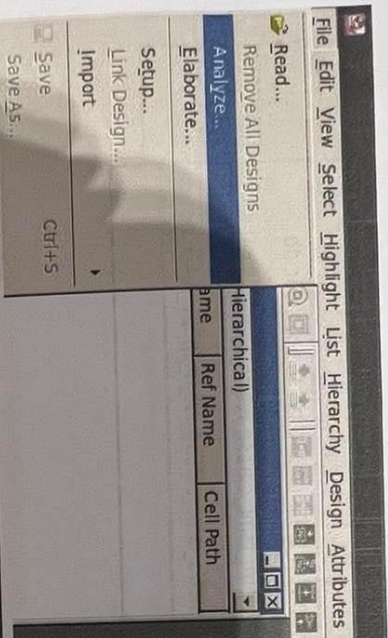


4. Reading RTL Files

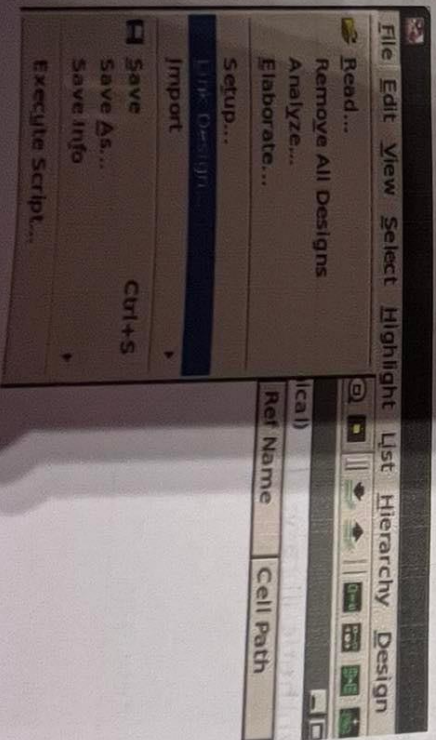
1) Method 1: Using Read Command



2) Method 2: Using Analyze & Elaborate commands

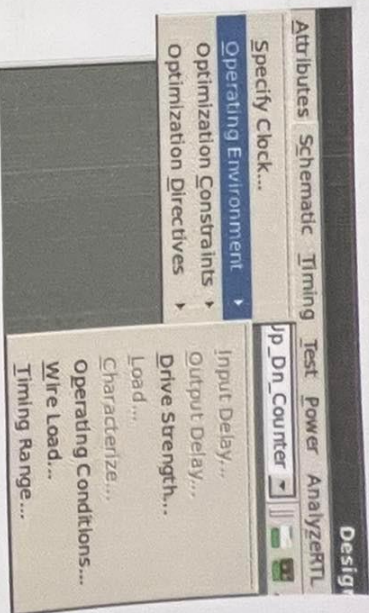
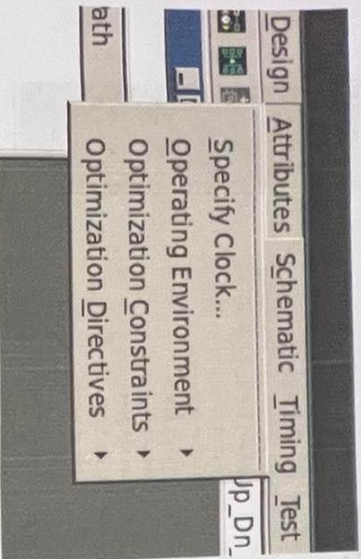


5. Link Design



6. Specifying different constraints (clocks, operating conditions, optimization)

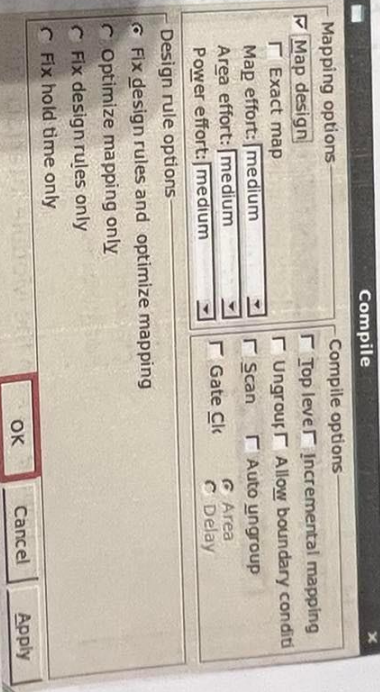
1) Select ... Attributes



7. Compile (Map & Optimize) RTL files

1) Select ... Design > Compile Design, Compile window will appear

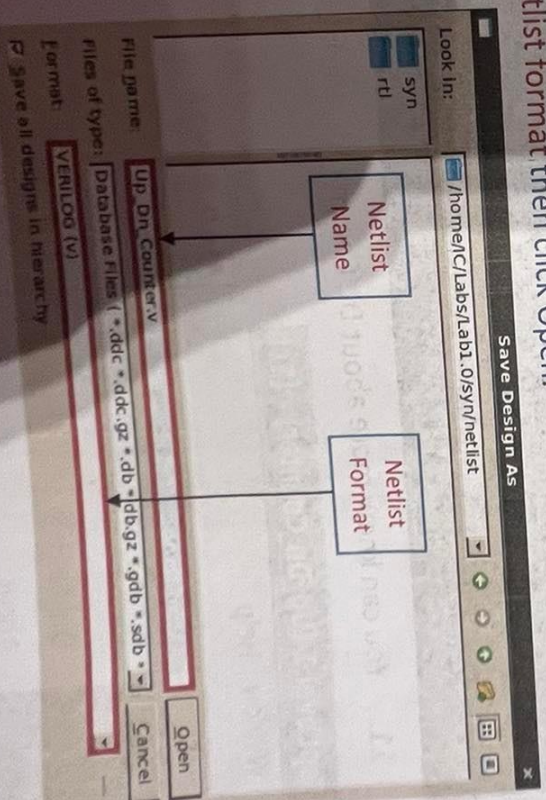
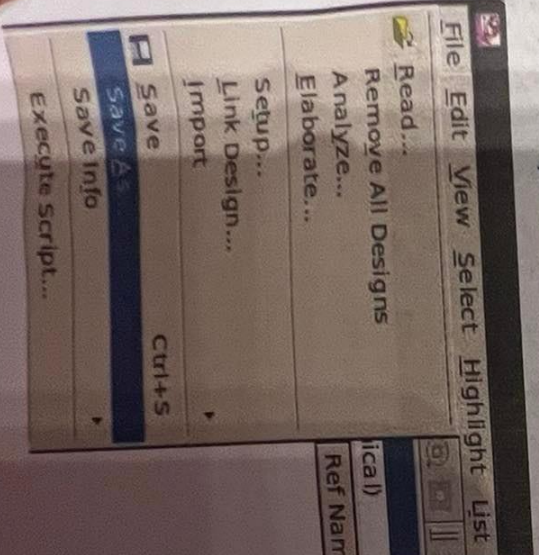
- Select your Mapping, Optimizing and compile option then Click OK



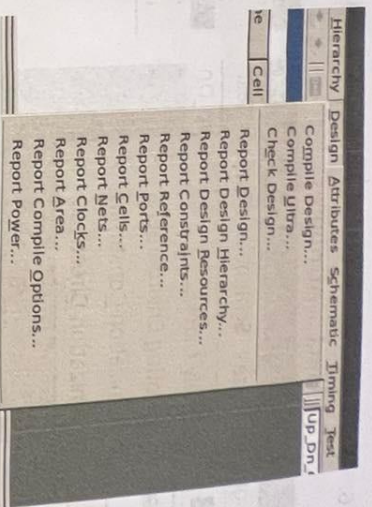
8. Generate gate level netlist

1) Select ... File > Save As, Save Design window will appear

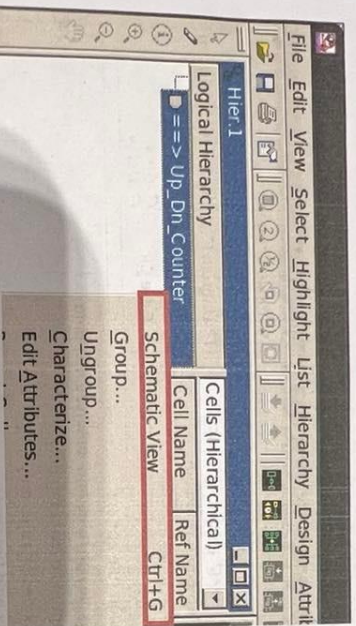
2) Select Netlist name and Netlist format then click Open.



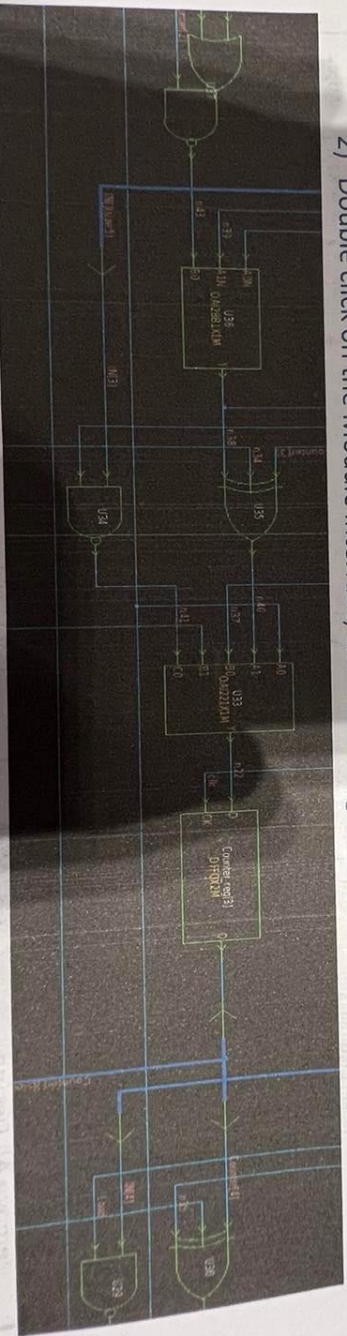
9. Generate different reports (power, area, constraints, clocks, ...)



10. Viewing Design Schematic & Hierarchy
- 1) Right Click on module and select Schematic View, Module Interface will appear



- 2) Double click on the Module interface, the module logic and connections will appear



11. You can know more about DC Commands, Variables and Error/Warning Messages

